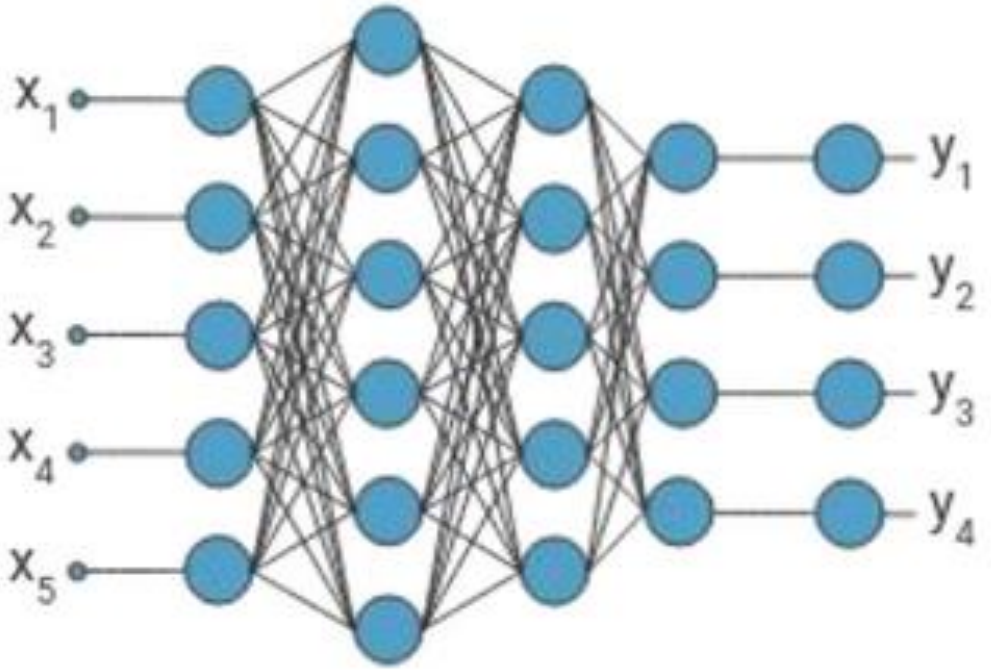
TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

INTELIGENCIA ARTIFICIAL

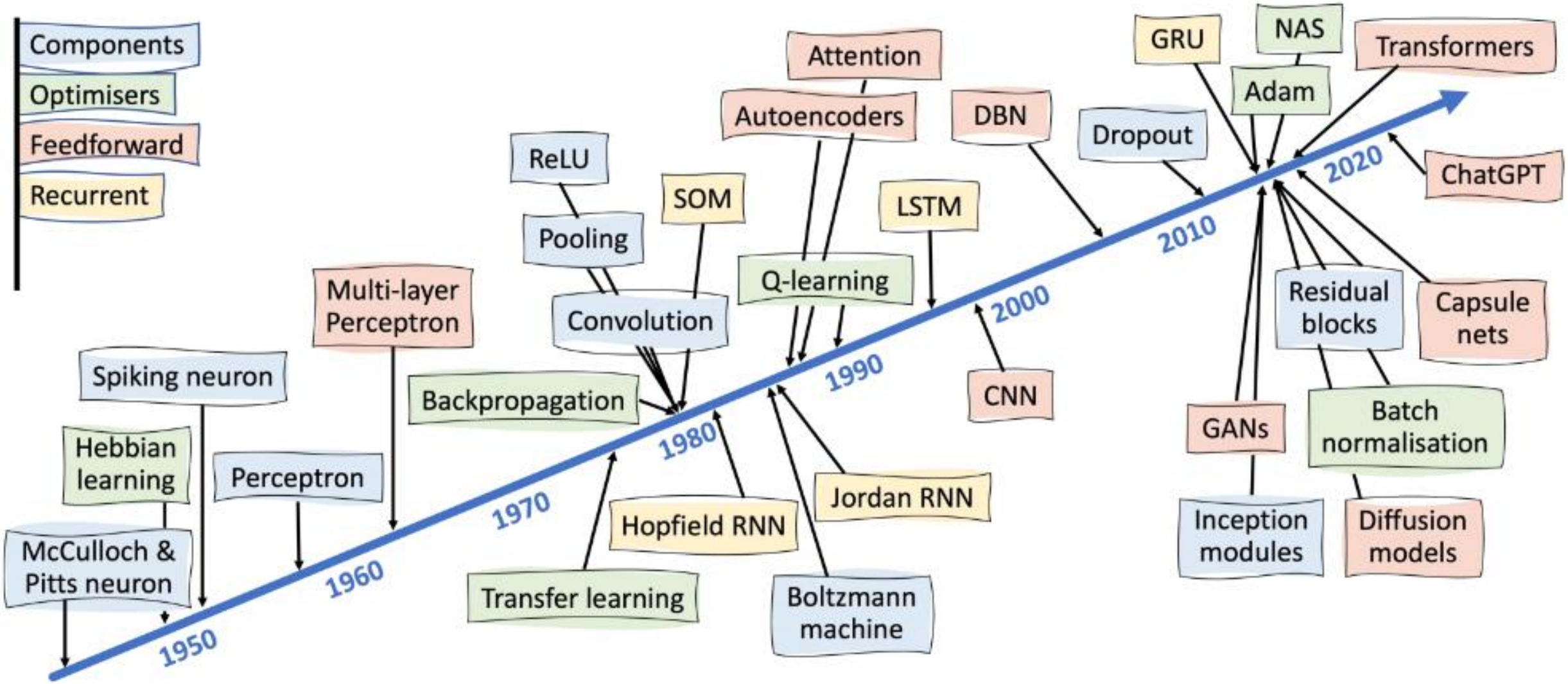
Prof. Me. Matheus Raffael Simon

REDES NEURAIS ARTIFICIAIS

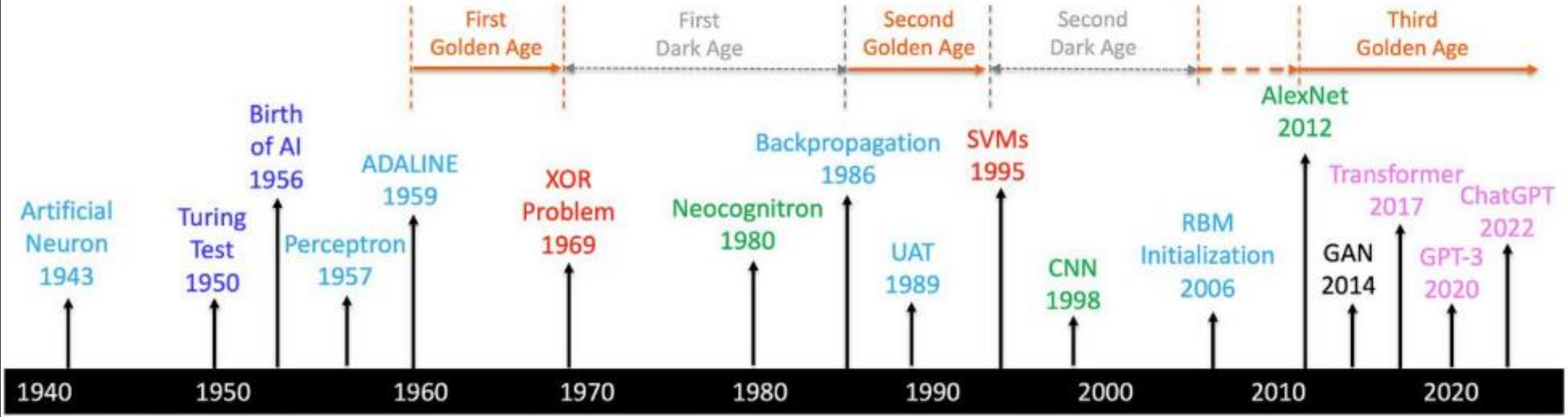
- Redes Neurais Artificiais são modelos de aprendizado de máquina “inspirados“ em redes de neurônios biológicos.
- analogia, metáfora tomada erroneamente (muitas vezes) como metonímia



História

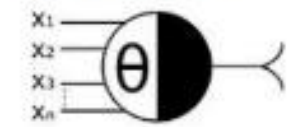

Components
Optimisers
Feedforward
Recurrent
McCulloch & Pitts neuron
Hebbian learning
Spiking neuron
Perceptron
Multi-layer Perceptron
Backpropagation
ReLU
Pooling
Convolution
SOM
Transfer learning
Hopfield RNN
Autoencoders
Attention
Q-learning
LSTM
Boltzmann machine
Jordan RNN
DBN
CNN
Dropout
GRU
NAS
Adam
Transformers
ChatGPT
Residual blocks
Capsule nets
GANs
Batch normalisation
Inception modules
Diffusion models
1950
1960
1970
1980
1990
2000
2010
2020

História



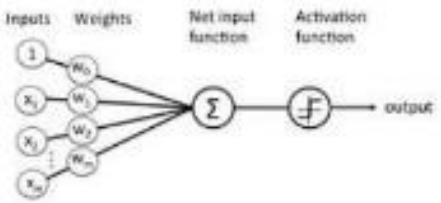


McCulloch-Pitts





Rosenblatt

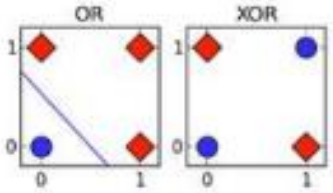




Widrow-Hoff

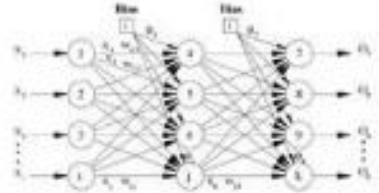


Minsky-Papert



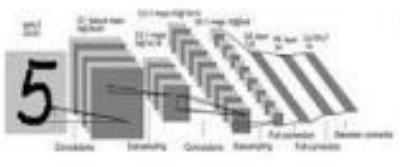


Rumelhart, Hinton et al.





LeCun

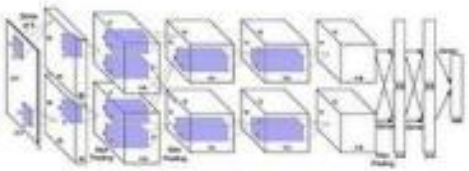




Hinton-Ruslan

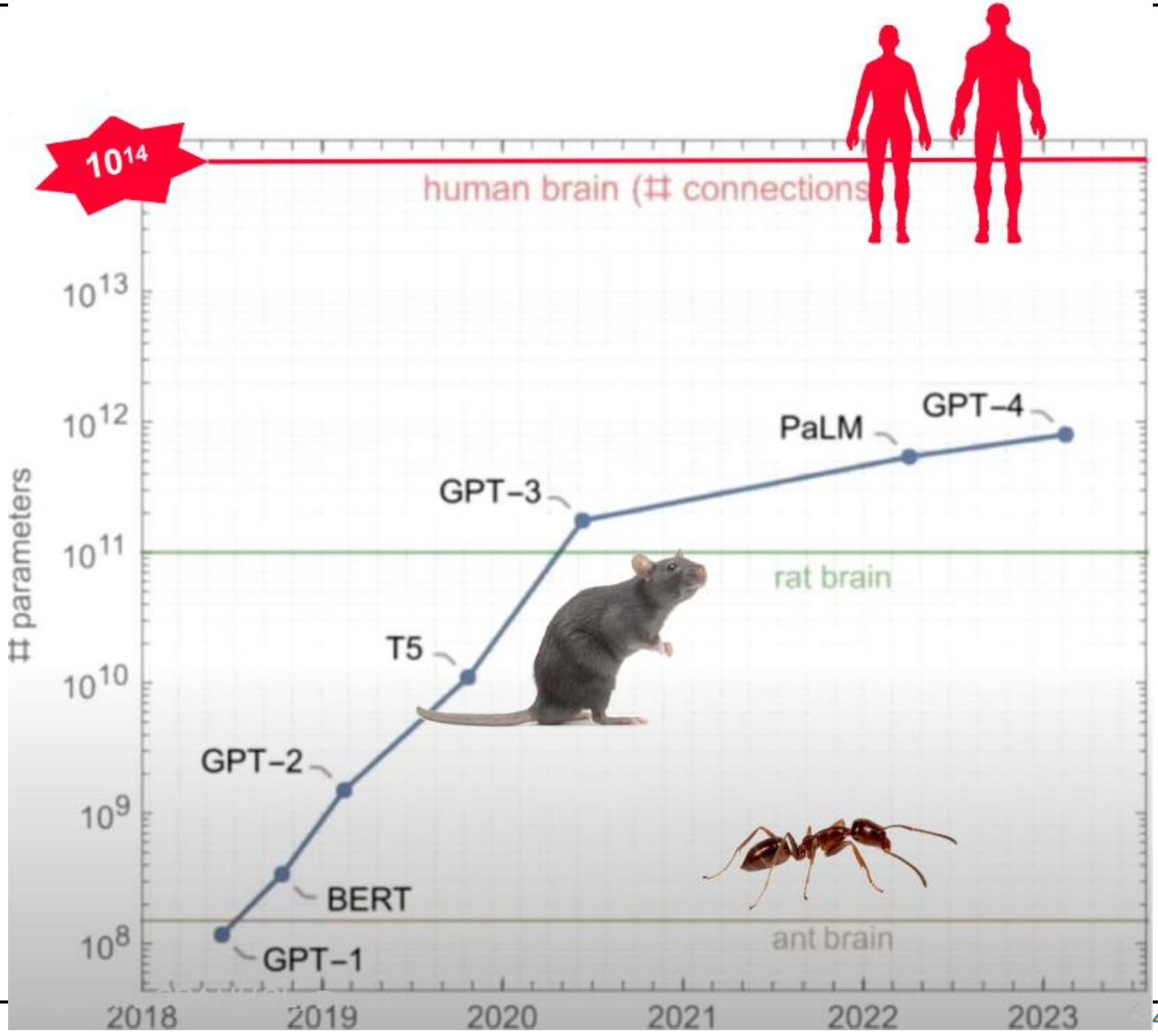


Krizhevsky et al.

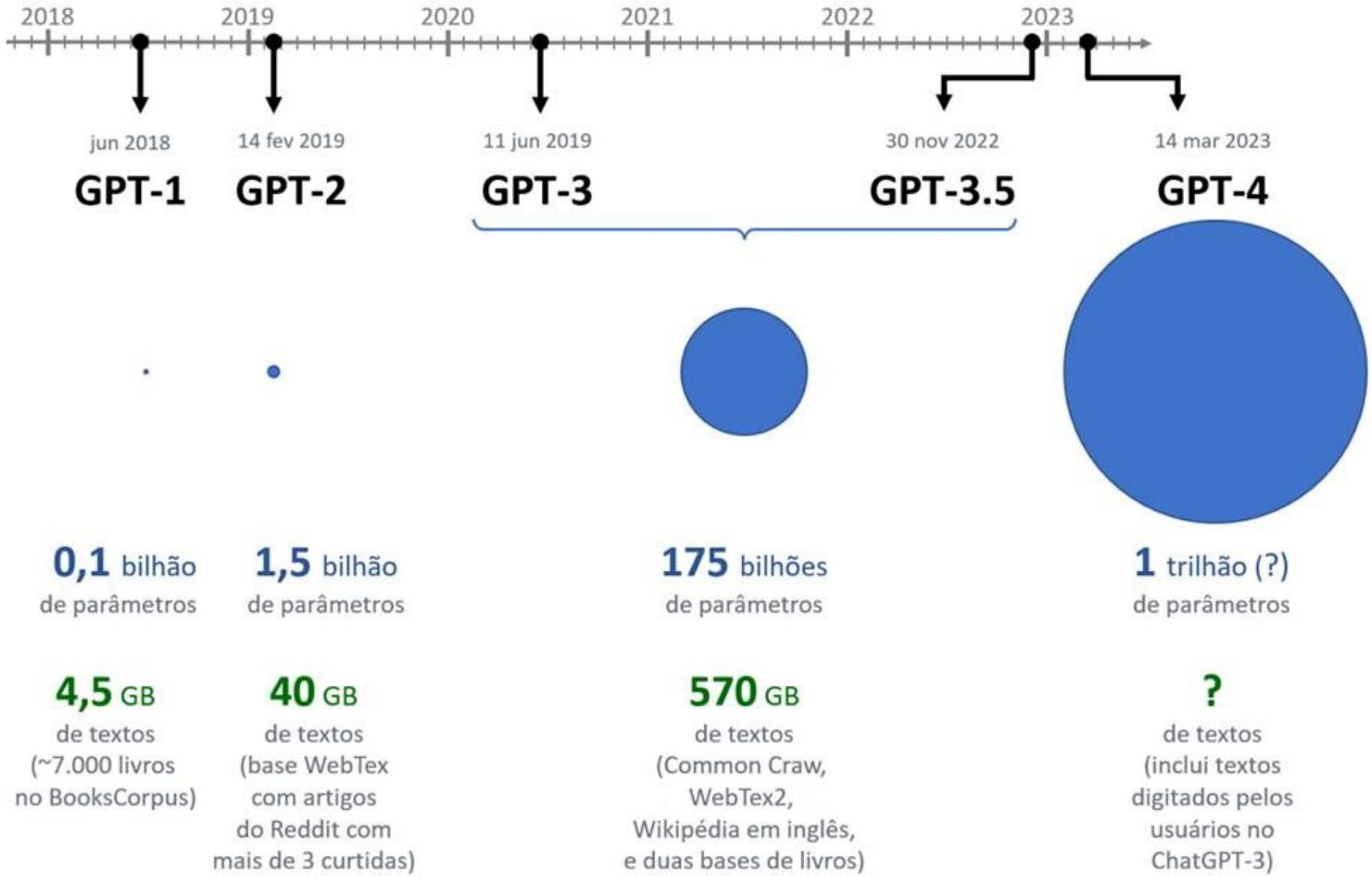




Vaswani

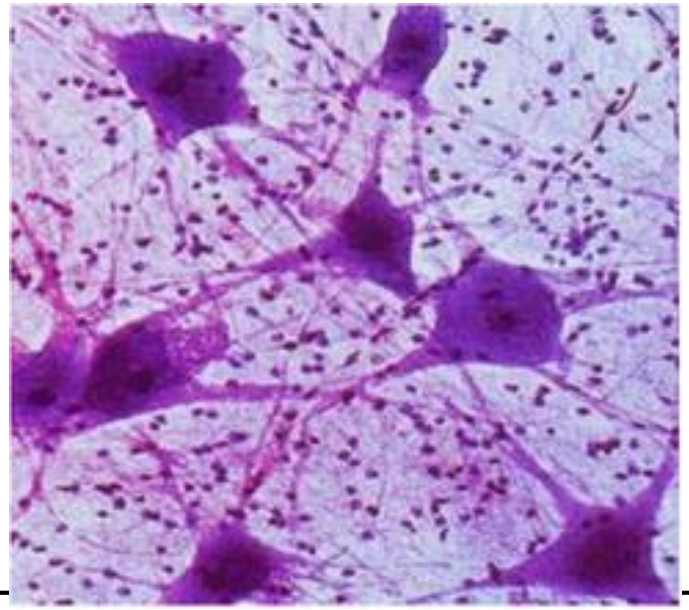

10^{14}
human brain (# connections)
10^{13}
10^{12}
10^{11}
10^{10}
10^{9}
10^{8}
parameters
GPT-4
PaLM
GPT-3
T5
GPT-2
BERT
GPT-1
rat brain
ant brain
2018
2019
2020
2021
2022
2023

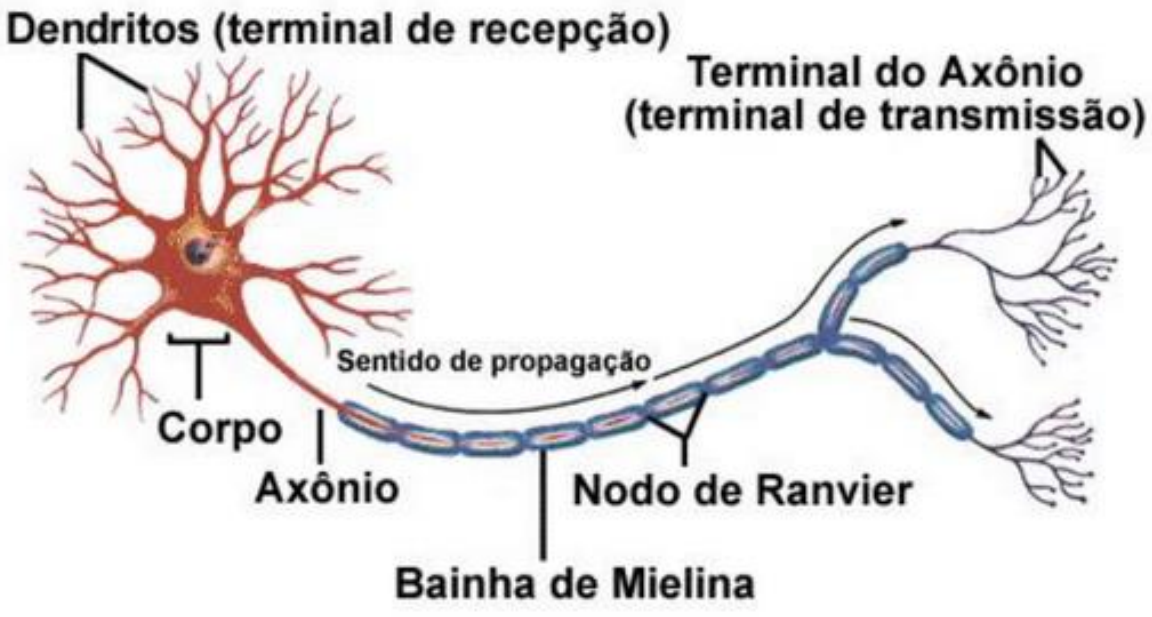
https://blog.saninternet.com/chatgpt



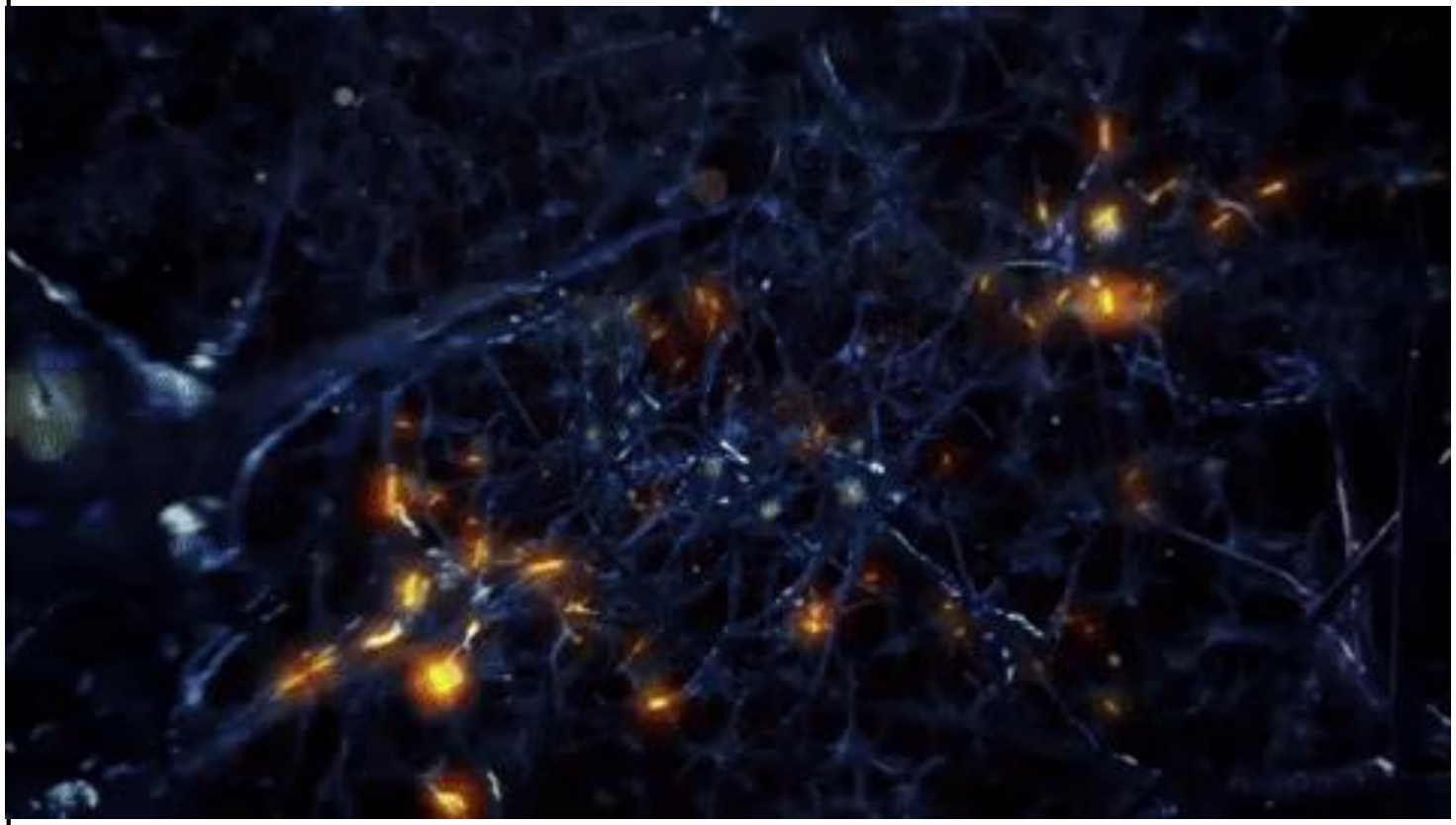
Neurônio biológico

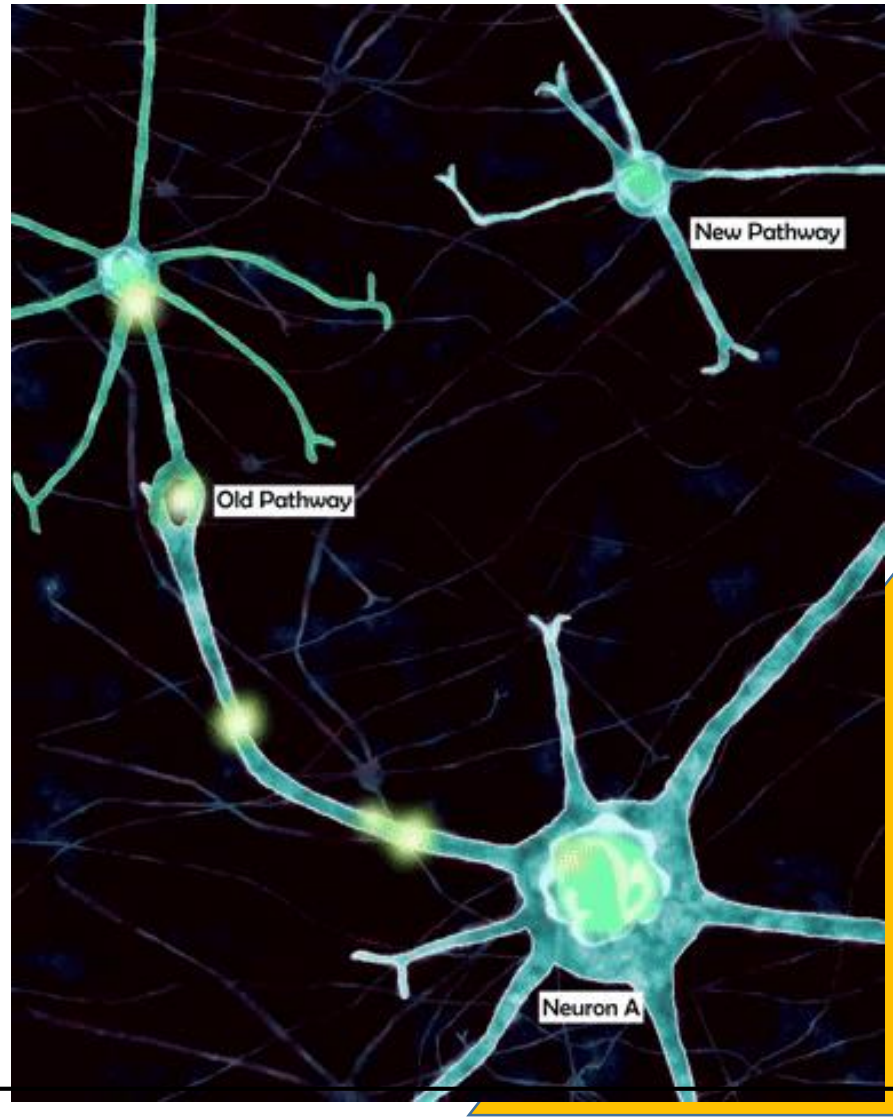
- Célula especializada em conduzir impulsos elétricos ~86 bilhões de neurônios no cérebro humano mais de 100 trilhões de conexões sinápticas
- Tecido nervoso (H&E) e representação esquemática de um neurônio



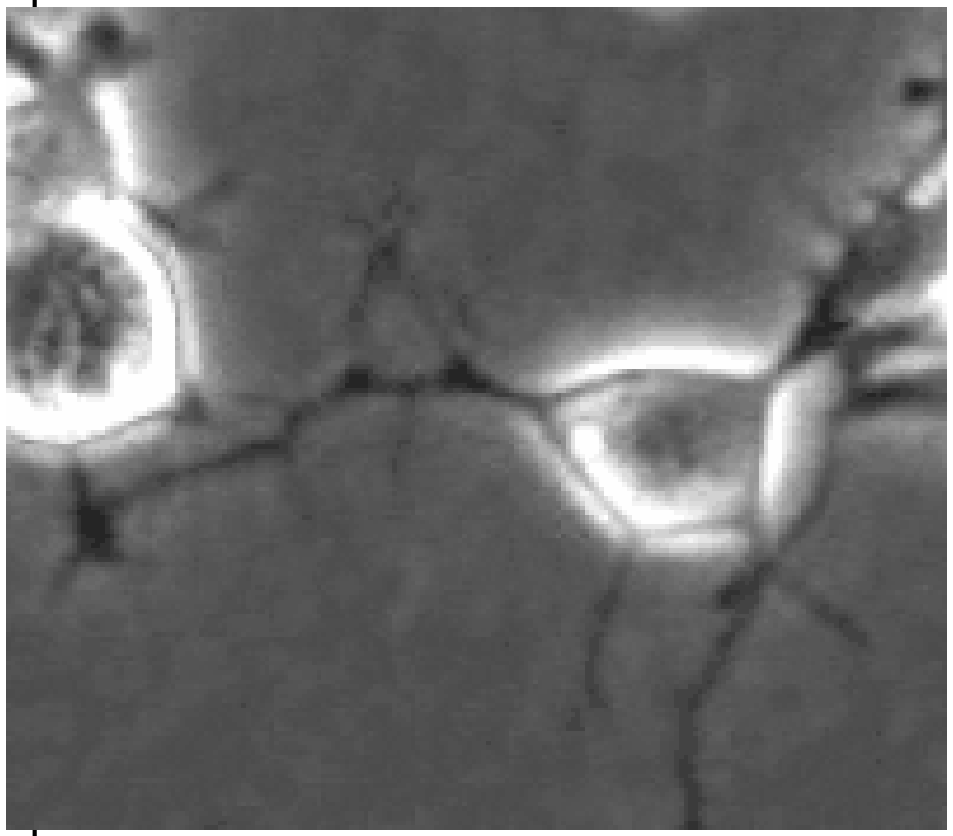


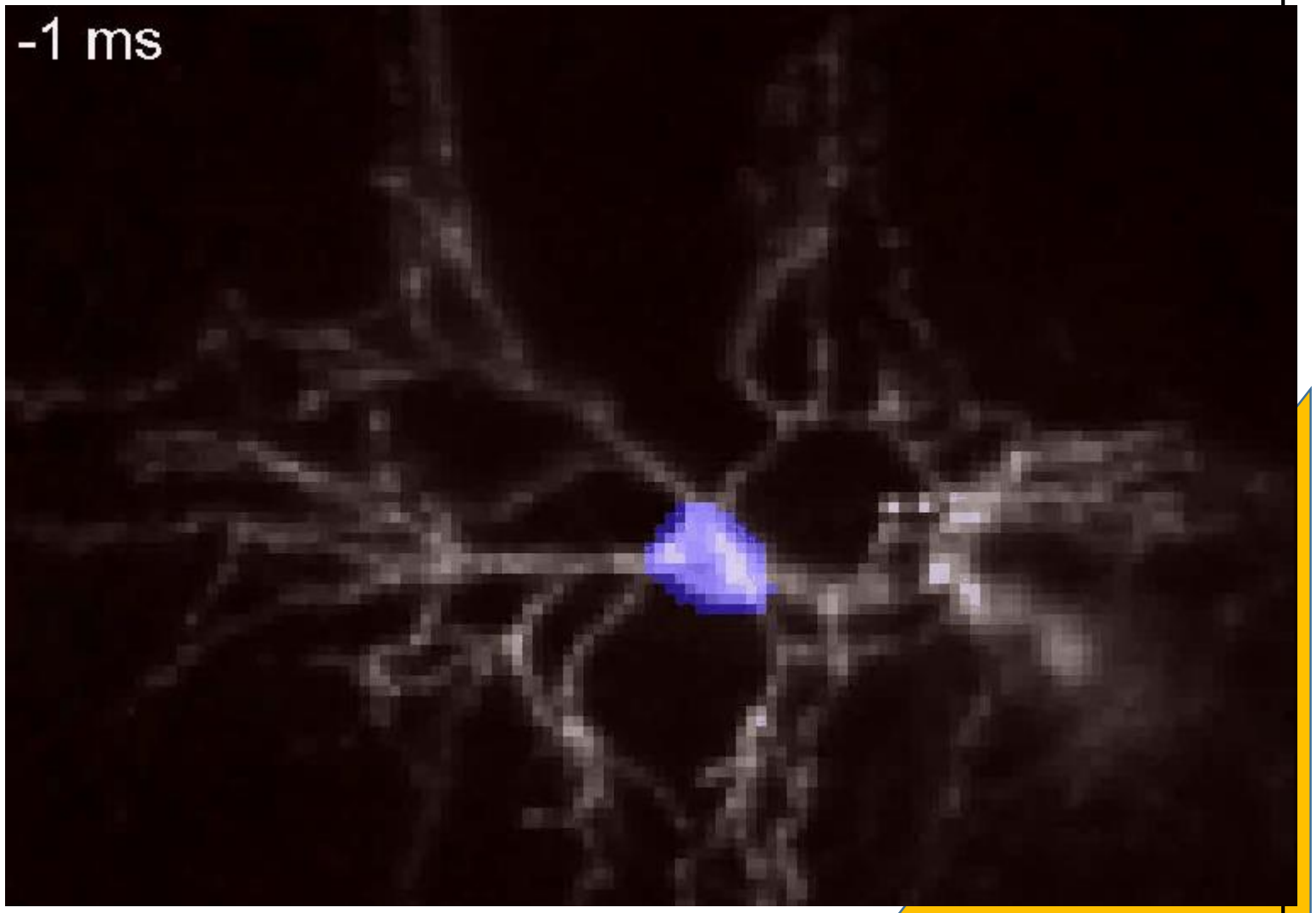
O que é um Neurônio real?

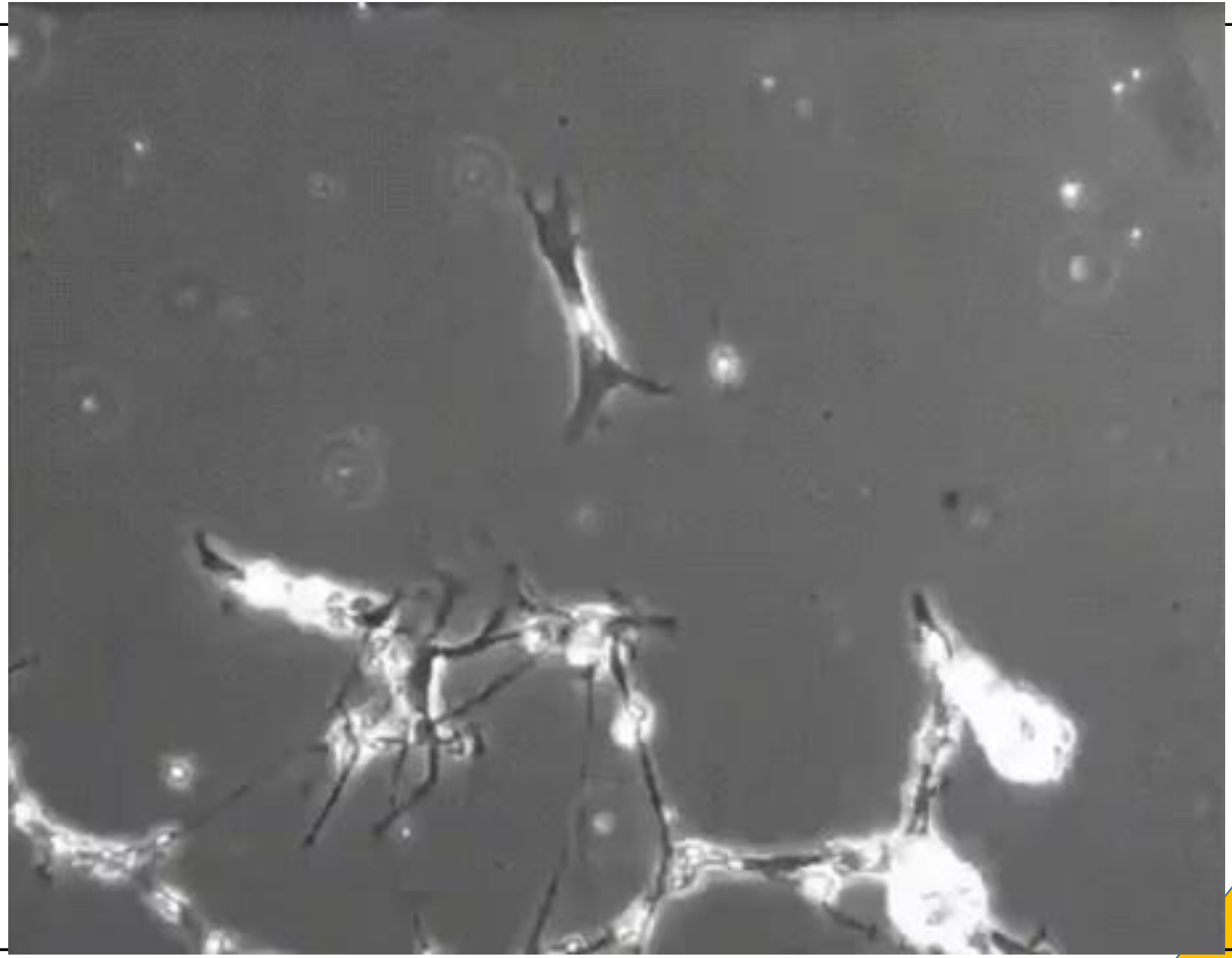




O que é um Neurônio real?







O que é um Neurônio real?

- Ver vídeo em anexo.

Neurônio artificial de McCulloch-Pitts

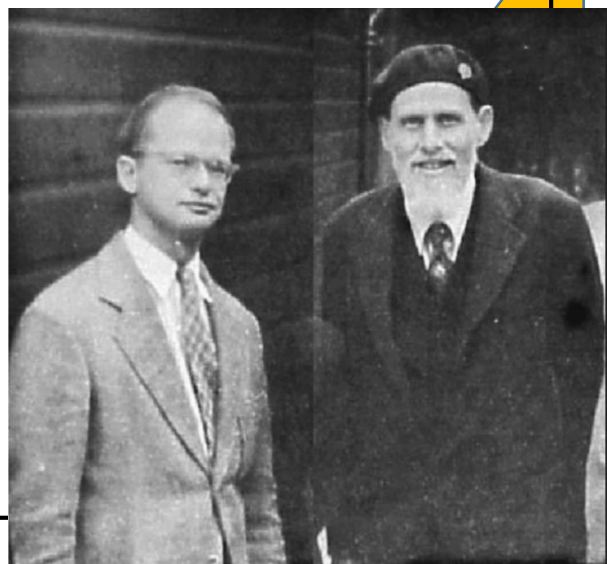
- 1943 - McCulloch e Pitts propuseram um modelo simples de neurônio artificial neuron
- uma ou mais entradas binárias e uma saída binária
- Função de ativação
- criar redes desses neurônios artificiais para computar proposições lógicas



Warren Sturgis McCulloch
(1898 – 1969)

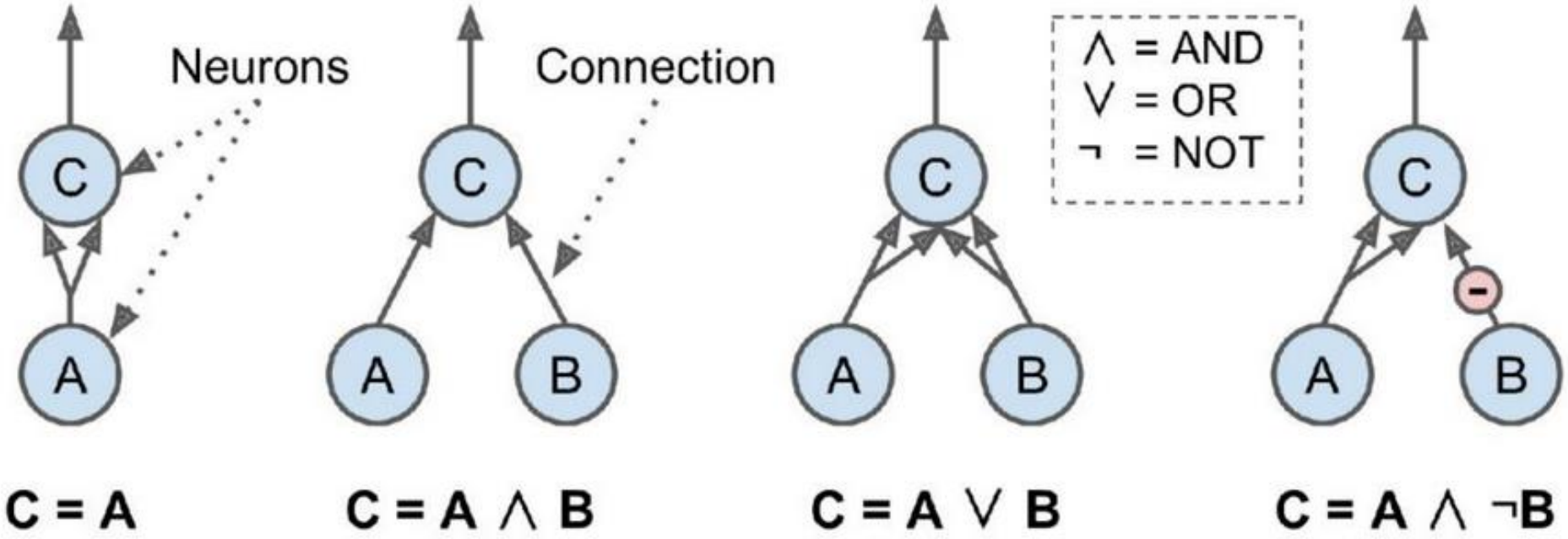


Walter Harry Pitts, Jr.
(1923 – 1969)



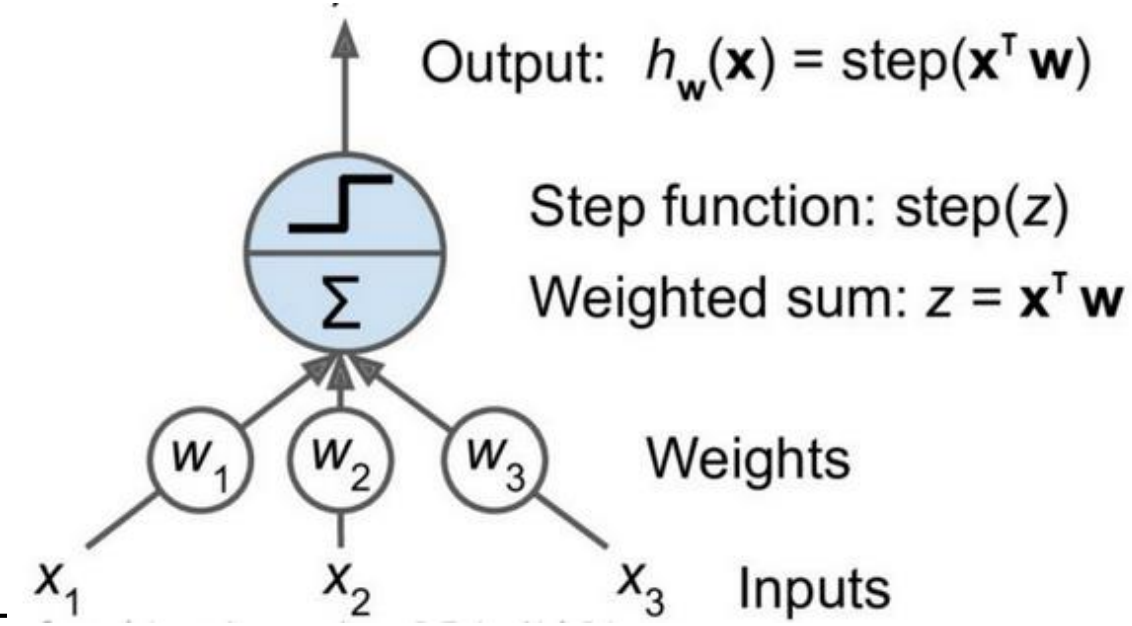
Neurônio artificial de McCulloch-Pitts

- Exemplos de ANNs simples. Um neurônio é ativado se pelos menos duas de suas entradas são ativadas.



Perceptron

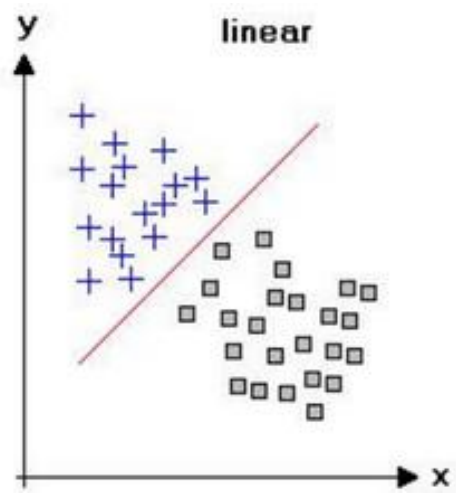
- 1957 - Frank Rosenblatt criou o Perceptron
- Mais simples ANN
- Threshold Logic Unit (TLU) > Entradas (com pesos associados) e saída são números

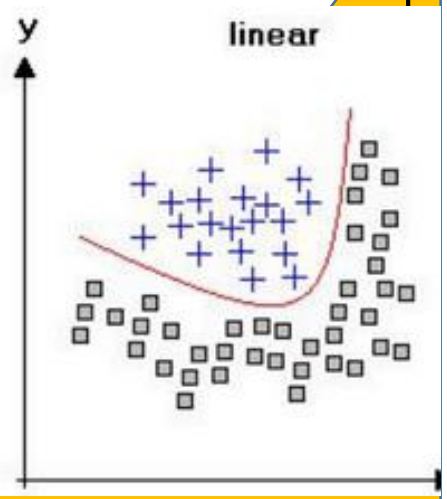




Perceptron

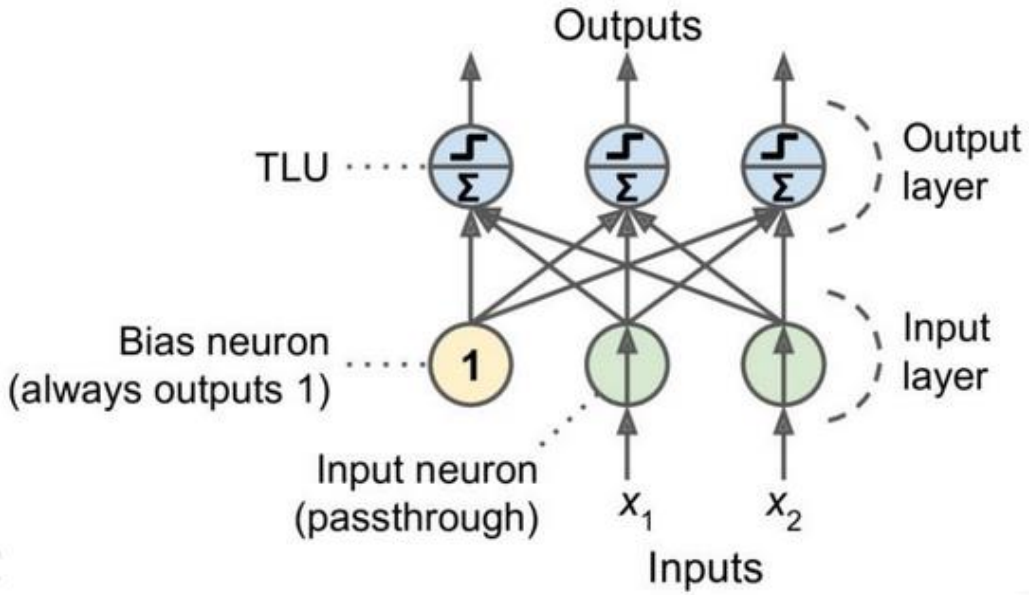
- O Perceptron é um classificador binário linear
- Se as instâncias de treino são linearmente separáveis converge para uma solução > Perceptron convergence theorem Computa uma combinação linear das entradas
- se o resultado excede um limiar, a saída é uma classe positiva
- caso contrário, classe negativa
- Similar a Logistic Regression ou Linear SVM





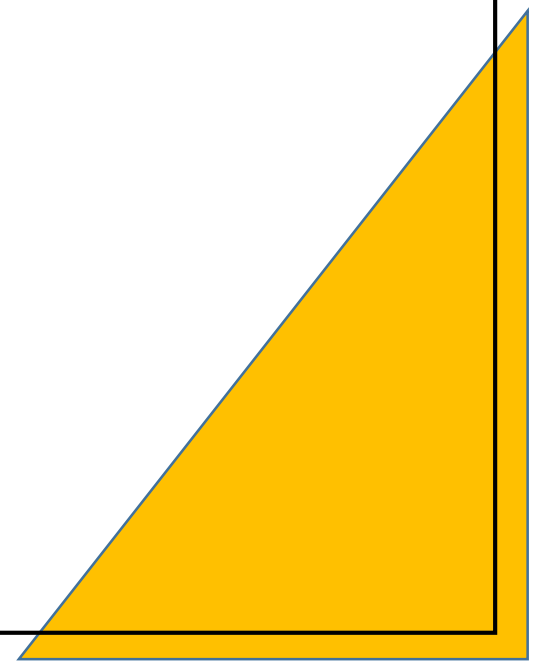
Perceptron: arquitetura

- Uma Perceptron é composta de uma única camada de TLUs conectadas a todas as entradas > fully connected layer ou dense layer
- Entradas são mapeadas por neurônios de entrada > input layer
- Bias > um neurônio especial > bias neuron (x0 = 1)



Perceptron: definição

- Matematicamente falando:
- $y=f(w \cdot x + b)$
- x é um vetor de entradas (inputs) formado por valores reais.
- w é o vetor de pesos (weights)
- b é o bias (valor pequeno, geralmente entre 0 e 1)
- f é a função de ativação (activation function)

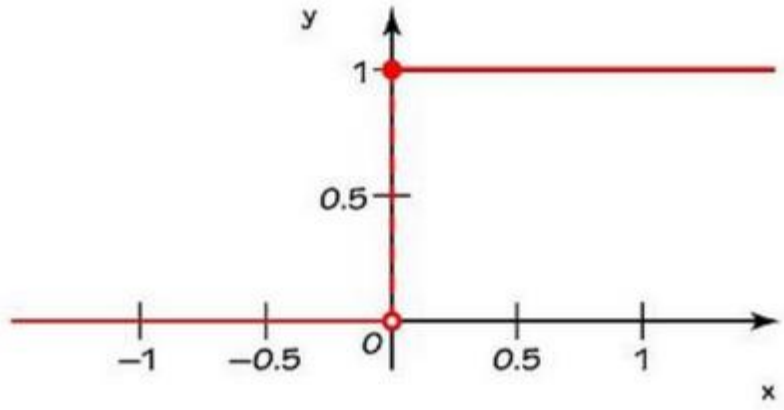


Perceptron: função de ativação

- A função de ativação f é uma função step ou threshold
- normalmente a Heaviside step function ou a sign.

$$\text{heaviside}\,(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases}$$

$$\text{sgn}\,(z) = \begin{cases} -1 & \text{if } z < 0 \\ 0 & \text{if } z = 0 \\ +1 & \text{if } z > 0 \end{cases}$$



Perceptron: Aprendizado

- Algoritmo de treinamento proposto por Rosenblatt e inspirado na regra de Hebb (1949) > quando um neurônio biológico dispara outro neurônio, frequentemente a conexão entre esses dois neurônios é fortalecida.
- Perceptrons são treinados usando uma variante dessa regra levando em consideração o erro cometido pela rede quando ela faz uma predição
- Treinar é encontrar os pesos certos que minimizem o erro > reforçar conexões

Perceptron: Algoritmo de aprendizado

- 1- Inicialize os pesos w e bias b para valores aleatórios pequenos.
- 2 - Para cada par entrada-saída (x, d) , compute a saída predita y fazendo f (w · x + b).
- 3 - Atualize os pesos e bias baseado no erro e=d−y fazendo,

w = w + η · e· x e b=b + η · e, sendo η a taxa de aprendizado (learning rate) > constante positiva

- 4 - Repita 1 e 3 por um **número fixo de iterações ou até o erro convergir**

Perceptron: Algoritmo de aprendizado

- 1- Inicialize os pesos w e bias b para valores aleatórios pequenos.
- 2 - Para cada par entrada-saída (x, d) , compute a saída predita y fazendo f (w · x + b).
- 3 - Atualize os pesos e bias baseado no erro e=d−y fazendo,

w = w + η · e· x e b=b + η · e, sendo η a taxa de aprendizado (learning rate) > constante positiva

- 4 - Repita 1 e 3 por um **número fixo de iterações ou até o erro convergir**

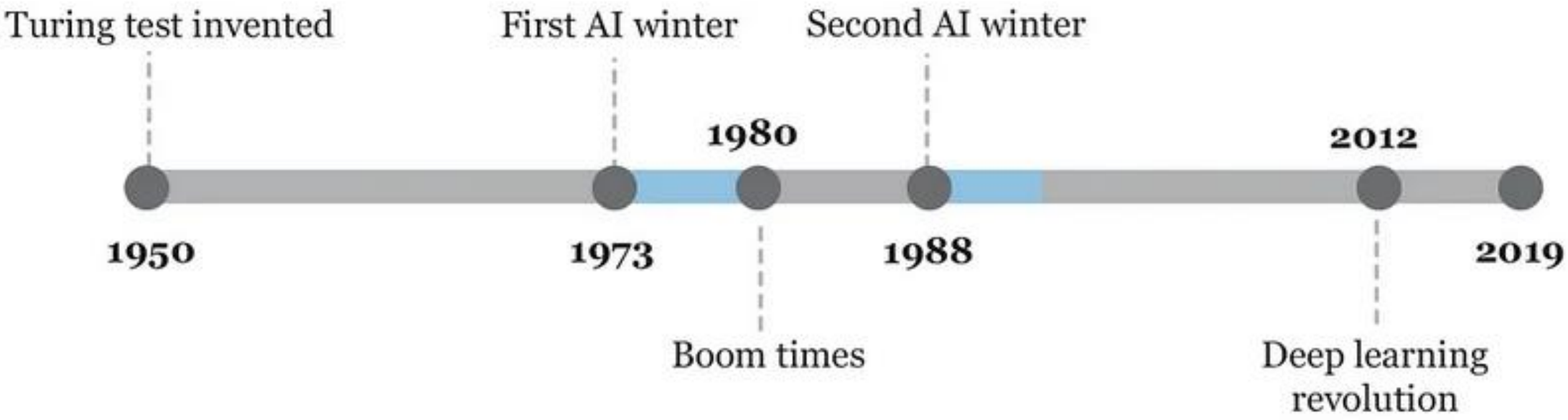

The Winter are Coming...

“Inverno da IA” / IA Winter

- Um "Inverno da IA" refere-se a um período de entusiasmo e financiamento significativamente reduzidos para pesquisa e desenvolvimento em inteligência artificial.
- É caracterizado por um declínio no interesse público e privado, levando a menos projetos de IA e a uma desaceleração do progresso da área.
- Esses invernos têm sido um fenômeno recorrente na história da IA, muitas vezes após períodos de intensa expectativa e investimento.

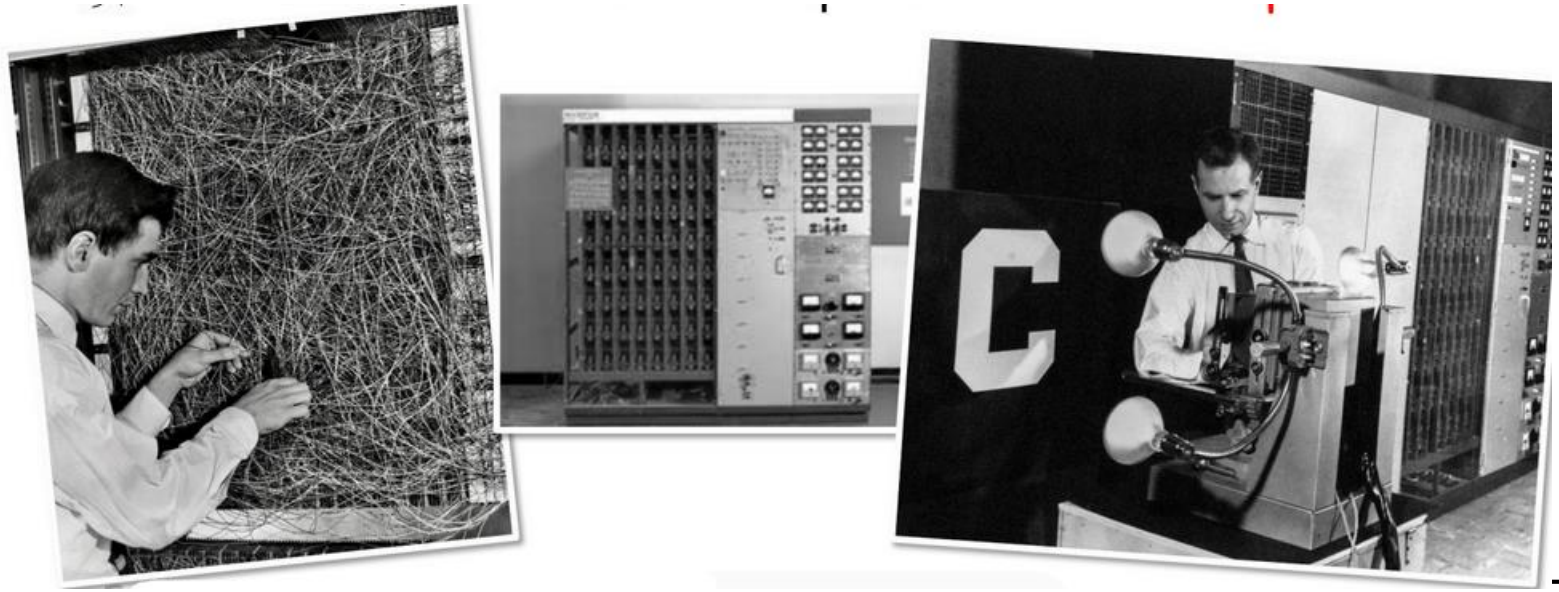
“Inverno da IA” / IA Winter

- Um



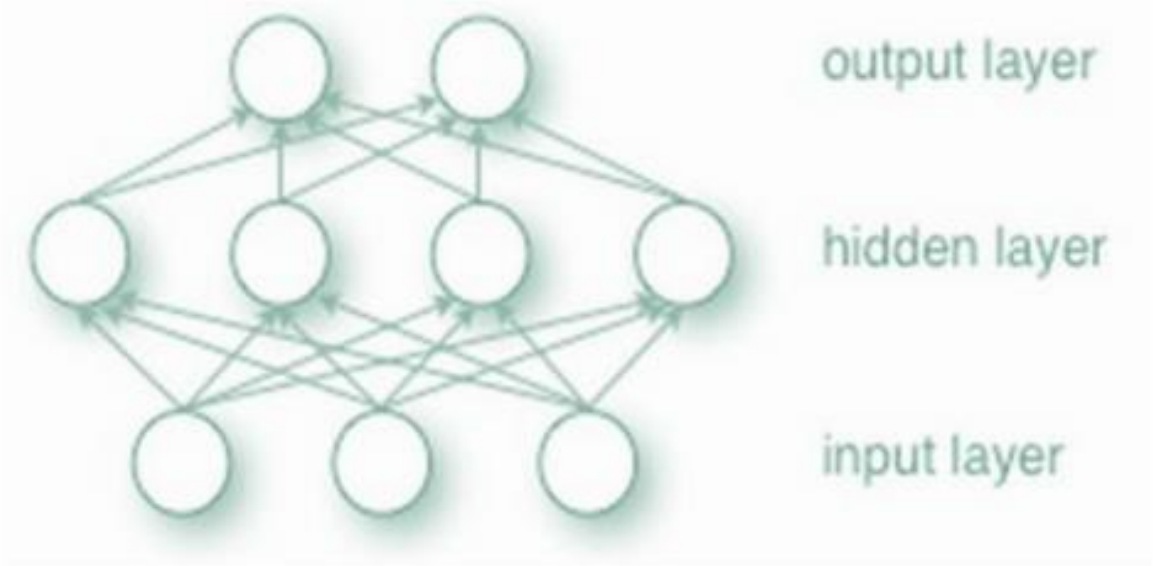
Afinal, o Perceptron..

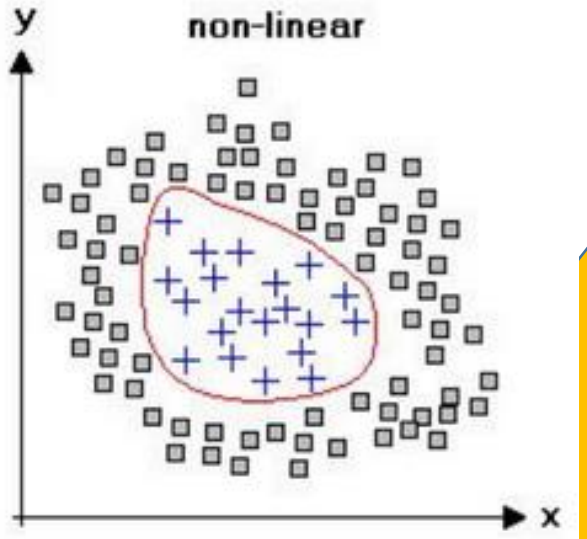
- Apesar das suas limitações fo a pedra fundamental das modernas ANNs
- Foi criado para ser uma máquina > Mark I Perceptron



Multilayer Perceptron (MLP)

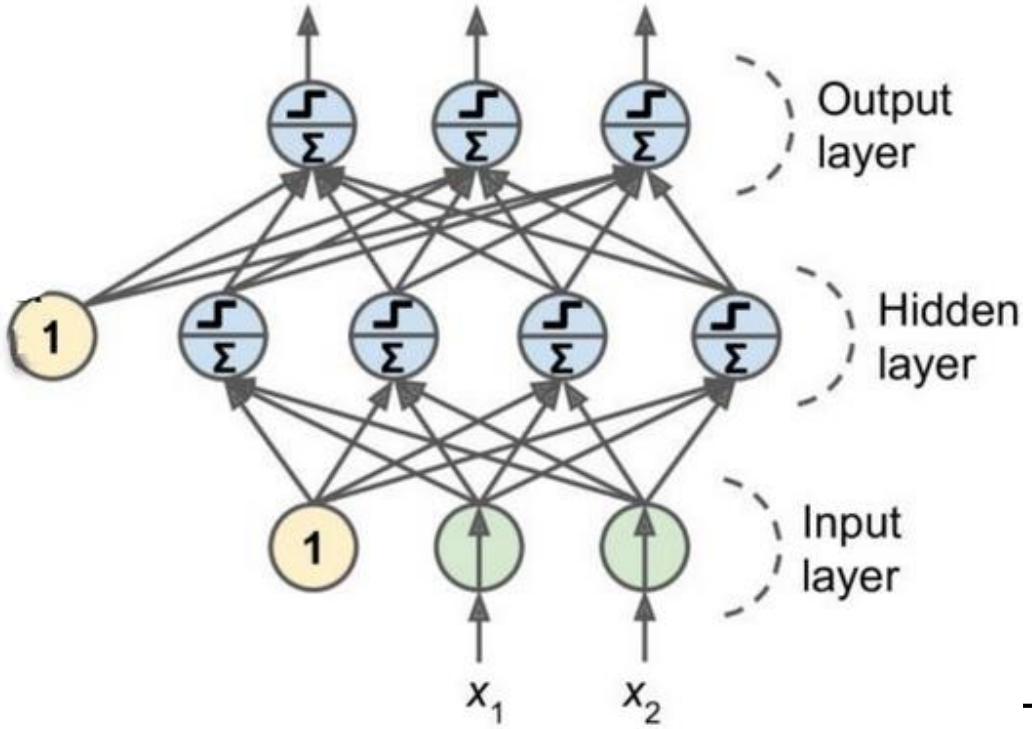
- Limitações dos Perceptrons podem ser eliminadas sobrepondo múltiplas camadas de Perceptrons Q Multilayer Perceptron (MLP)
- Uma NN multicamada é um classificador não-linear





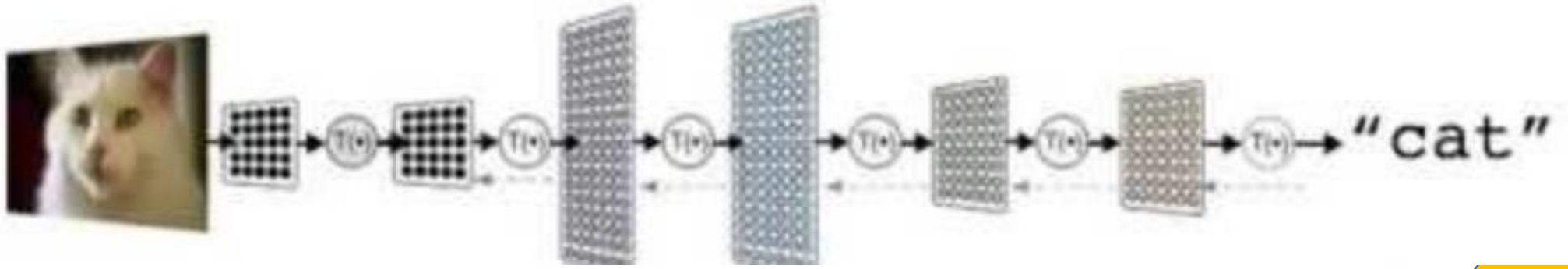
MLP= Arquitetura

- Cada camada é fully connected com a próxima inclui um bias, exceto a última camada
- O sinal segue apenas uma direção Q Feedforward NN (FNN)



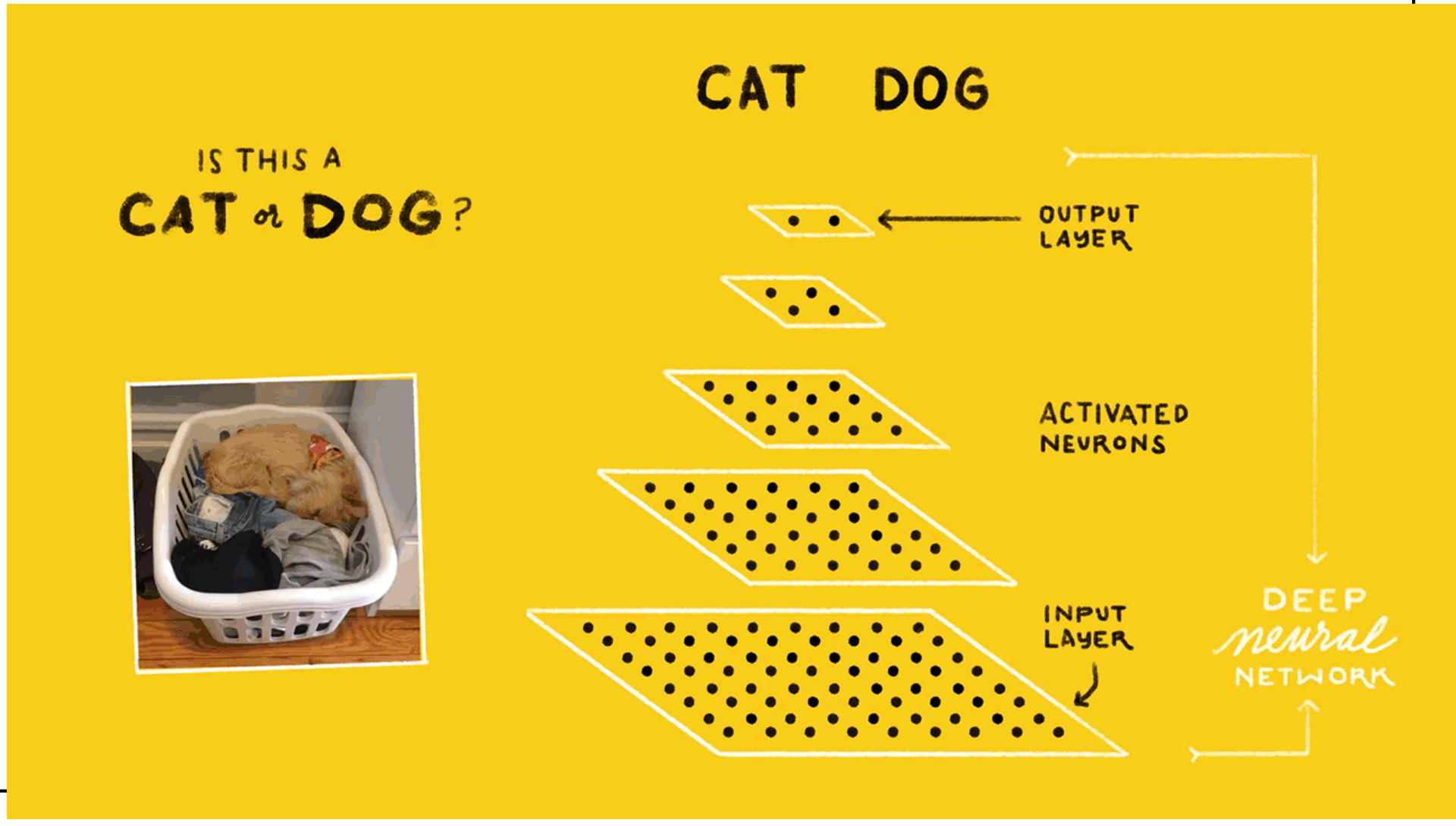
Deep NN

- DNN é uma hierarquia de várias camadas ocultas. Cada camada transforma os dados de entrada em representações mais abstratas
- Ex.: contornos > nariz Q face
- Camada final combina as representações > Predição



Deep NN

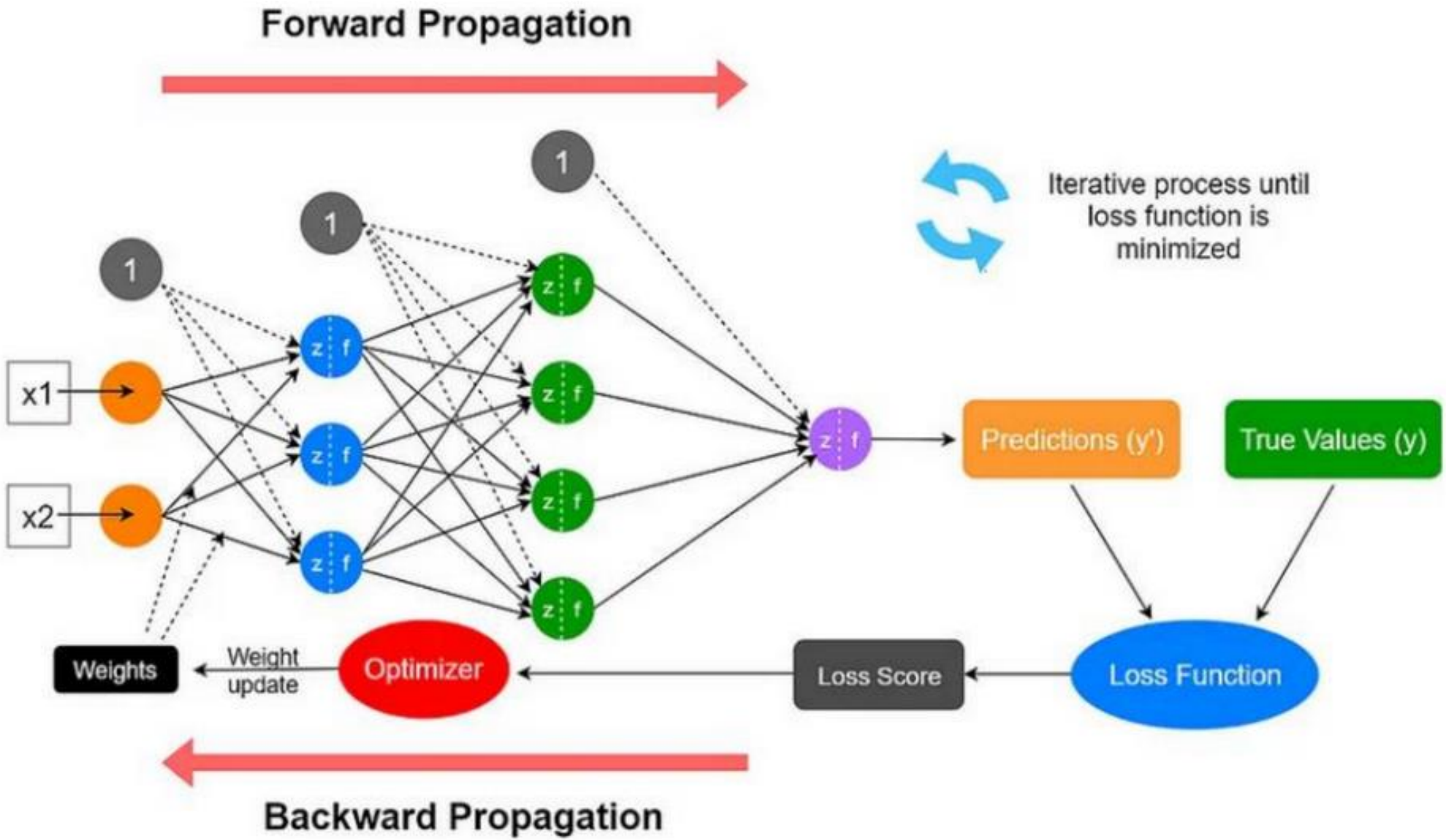
- Atualmente, se uma NN tem mais que 2 camadas ocultas ela é profunda!
- Mas, podem existir centenas de camadas!



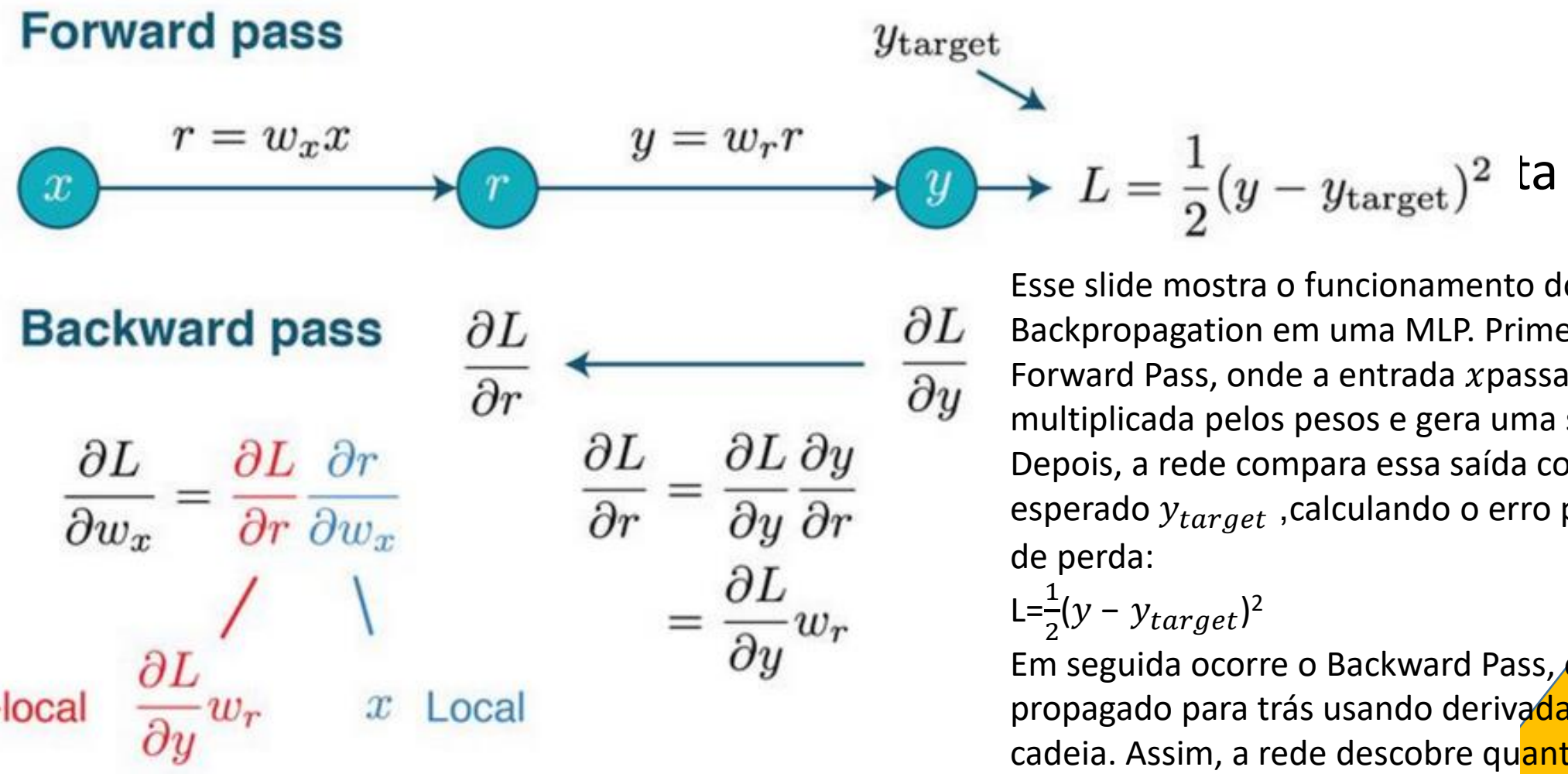
Treinando uma MLP



Treinando uma MLP



MLP: Backpropagation



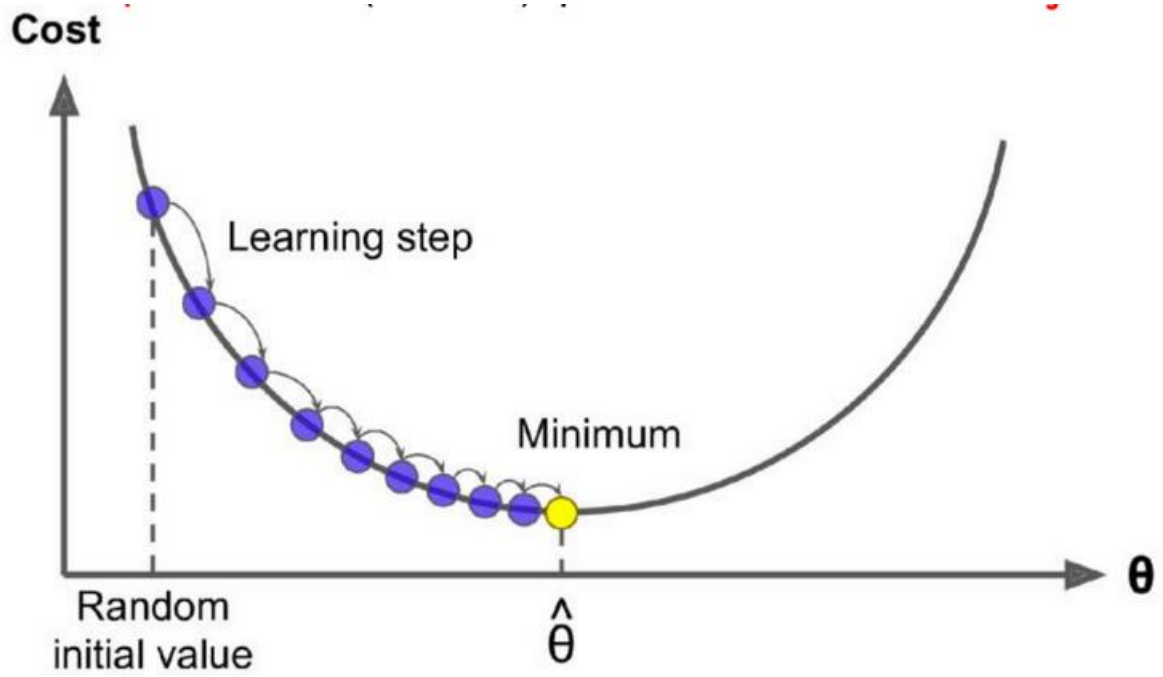
Esse slide mostra o funcionamento do Backpropagation em uma MLP. Primeiro ocorre o Forward Pass, onde a entrada $x$passa pela rede, é multiplicada pelos pesos e gera uma saída y. Depois, a rede compara essa saída com o valor esperado y_{target} ,calculando o erro pela função de perda:

$L=\frac{1}{2}(y - y_{target})^2$

Em seguida ocorre o Backward Pass, onde o erro é propagado para trás usando derivadas e a regra da cadeia. Assim, a rede descobre quanto cada peso contribuiu para o erro e ajusta os pesos para melhorar os resultados durante o treinamento.

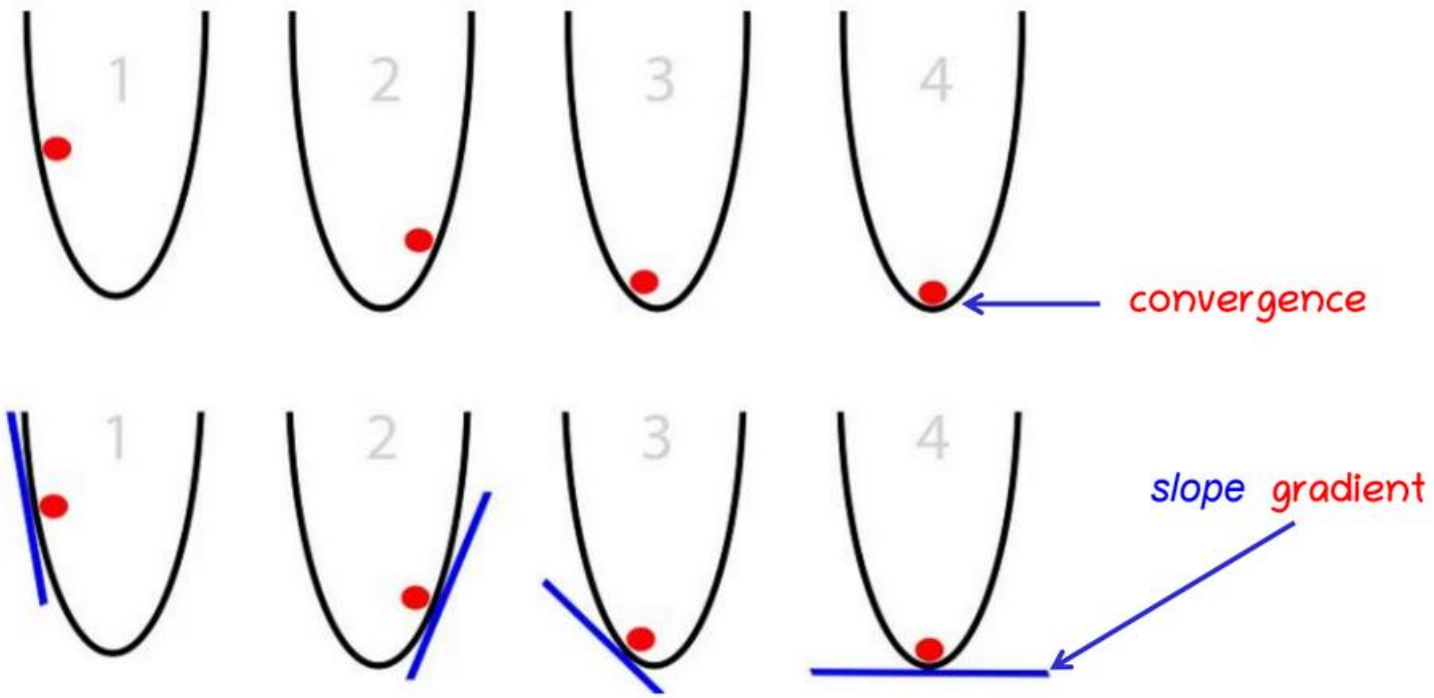
Interlúdio: Gradiente Descent (GD)

- Backpropagation é uma variação do algoritmo Gradient Descent
- ajuste iterativo de parâmetros (vetor θ) para minimizar uma função de custo



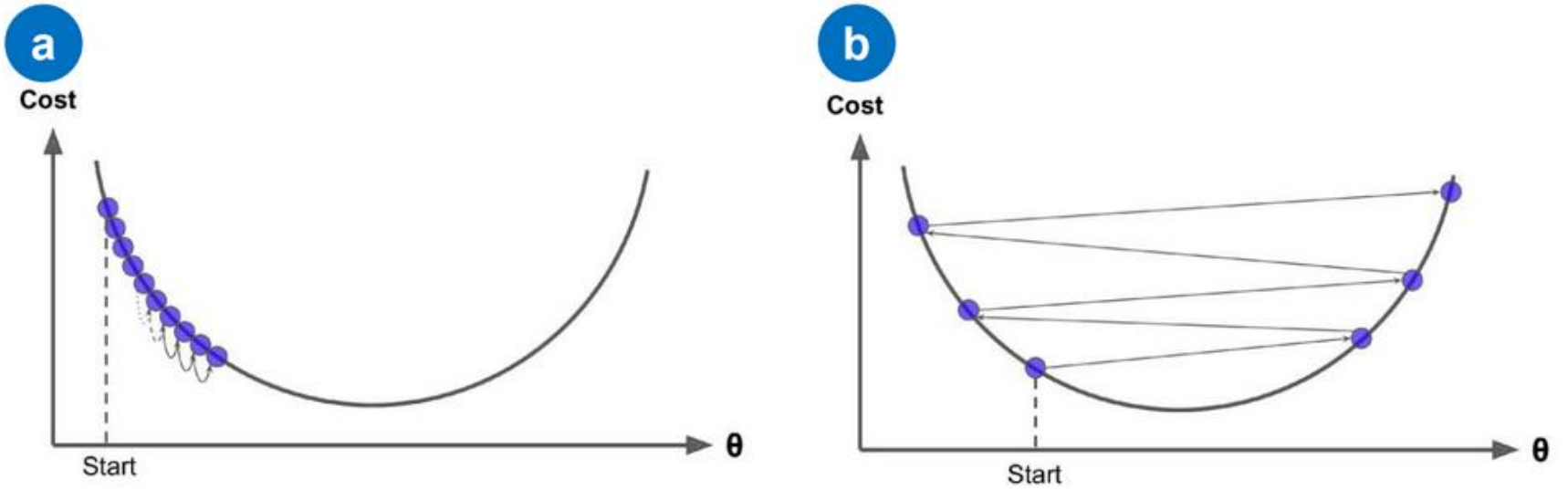
Gradiente Descent (GD)

- GD: Otimização



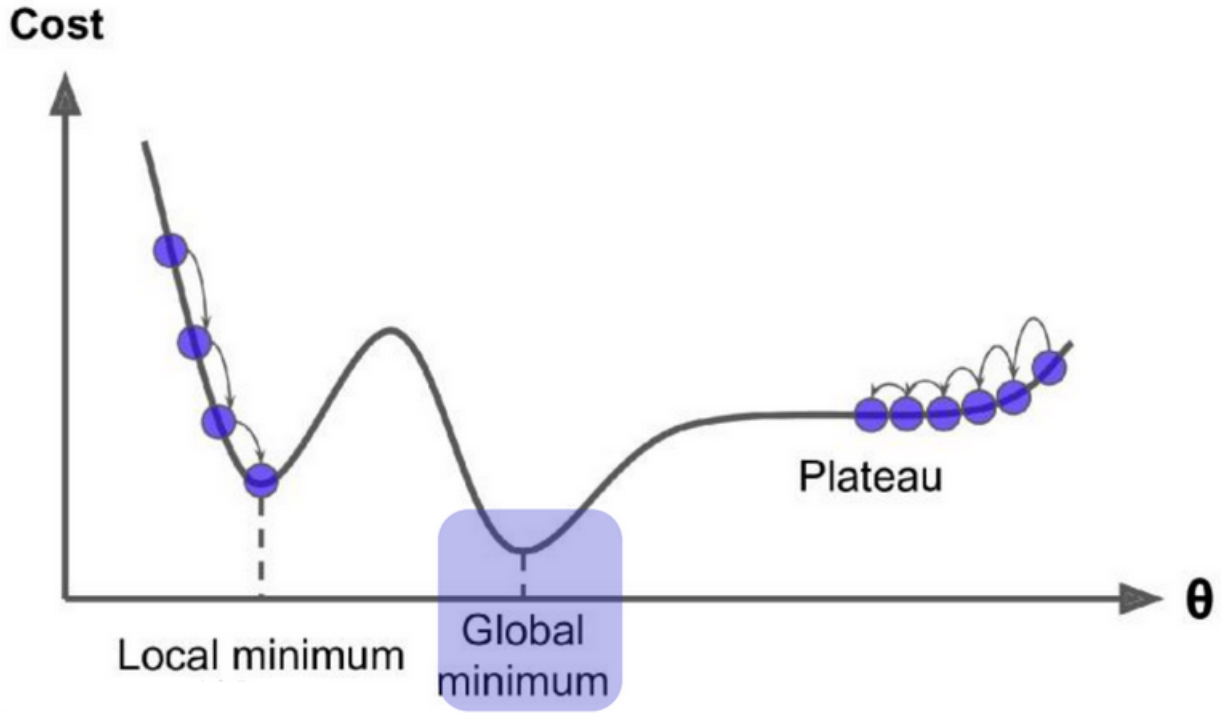
Gradiente Descent (GD) Learning Rate

- O hyperparâmetro learning rate determina o **tamanho dos passos**
- (a) muito **pequeno** > muitas iterações, mais tempo para convergir
- (b) muito **grande** > algoritmo diverge



Gradiente Descent (GD) Funções desafiadoras

- Inicialização randômica pode levar a:
- Mínimo local
- Platô

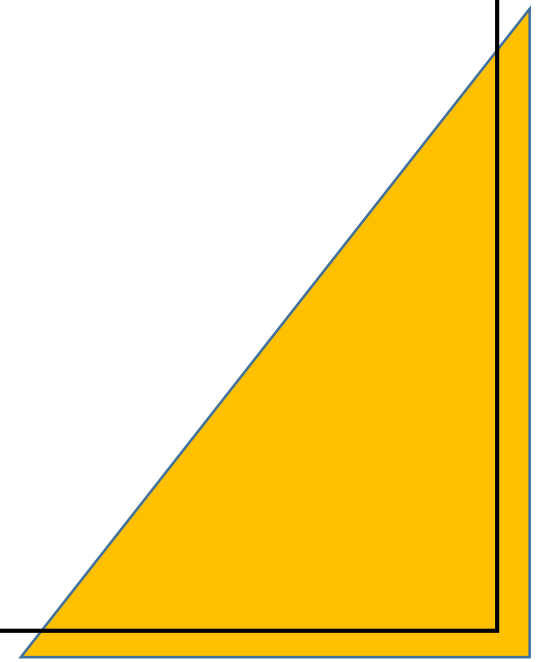


Gradiente Descent (GD) Tipos

- **Batch GD >** usa todo o dataset de treino
- computacionalmente caro
- suavemente converge

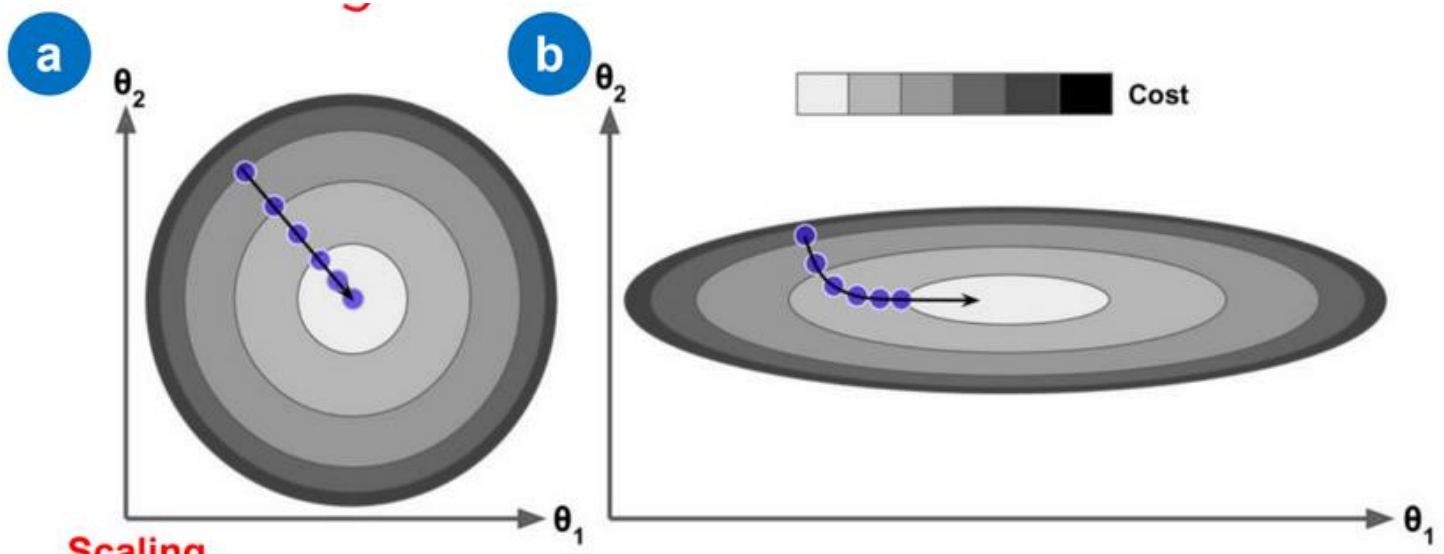
- **Stochastic GD** > usa uma instância randômica
- mais rápido
- menos regular, podendo chegar em resultado não ótimo

- **Mini-Batch GD** > meio termo, pequeno subconjunto
- computacionalmente eficiente



GD: Batch

- Scaling (escala)> É importante garantir que todas as features têm escala similar > evitar um longo tempo para convergir



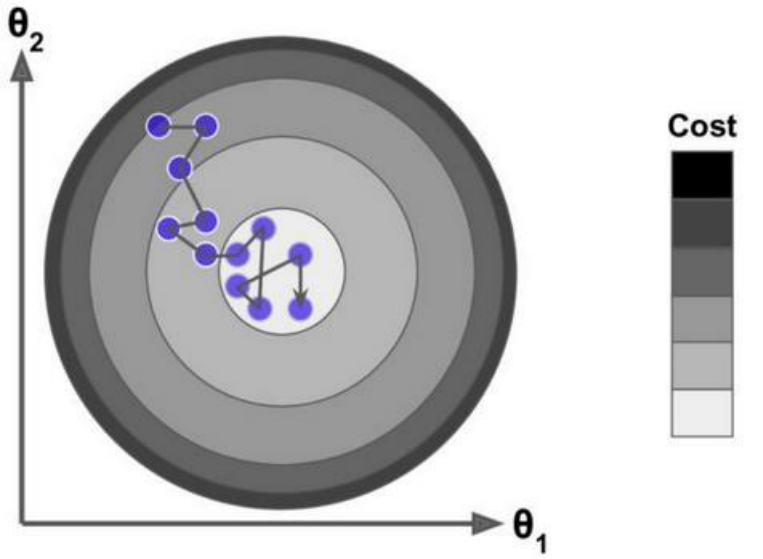
Scaling

(a) *Features* com mesma escala.

(b) *Features* com escalas diferentes: 1 é muito menor que 2.

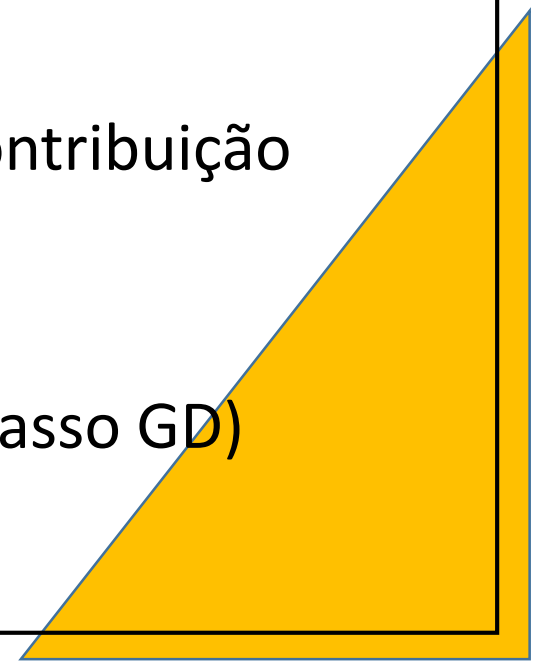
GD: Stochastic

- Uma instância randômica selecionada em cada passo:
- learning schedule > gradualmente reduzir a taxa de aprendizado
- Cada round de iterações é uma epoch



MLP: Backpropagation

- Utiliza mini-batch (32 instâncias, p.ex.)
- Para cada instância de treinamento, o algoritmo de **backpropagation** primeiro faz uma predição (forward pass) e mede o erro > **loss function**
- Em seguida, percorre cada camada em **reverso** para medir a contribuição de erro de cada conexão (**reverse pass**)
- Finalmente ajusta os pesos das conexões para reduzir o erro (passo GD)

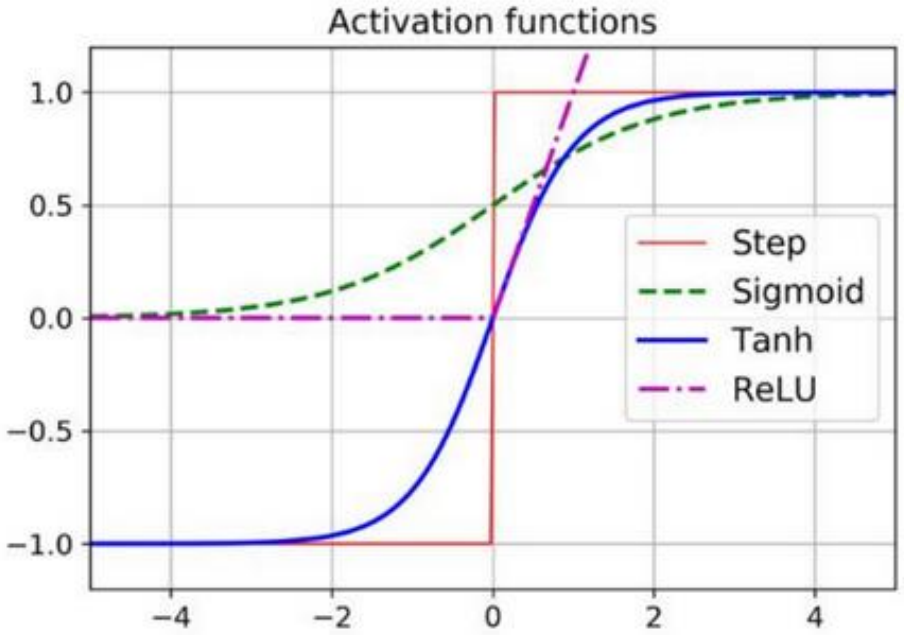


MLP: Backpropagation

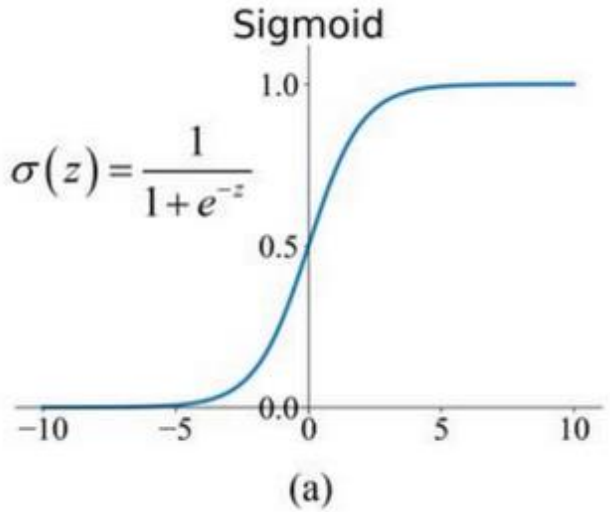
- A função step foi substituída pela função logística (**sigmoid**)

$$f(x) = \frac{1}{1 + e^{-x}}$$

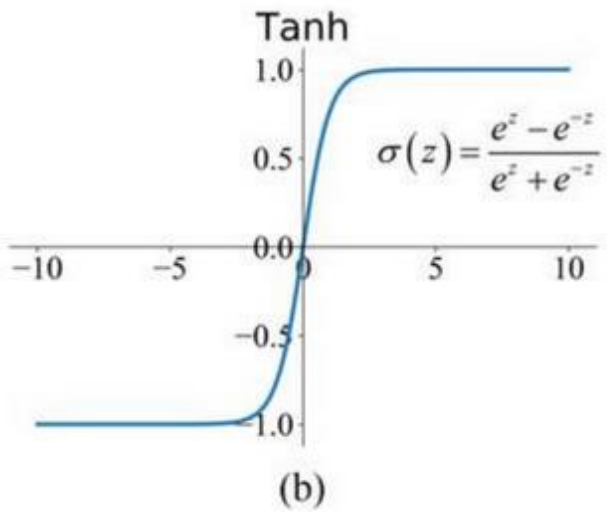
- Outras são possíveis: Hyperbolic tangent (**Tanh**), Rectified Linear Unit (**ReLu**), etc.



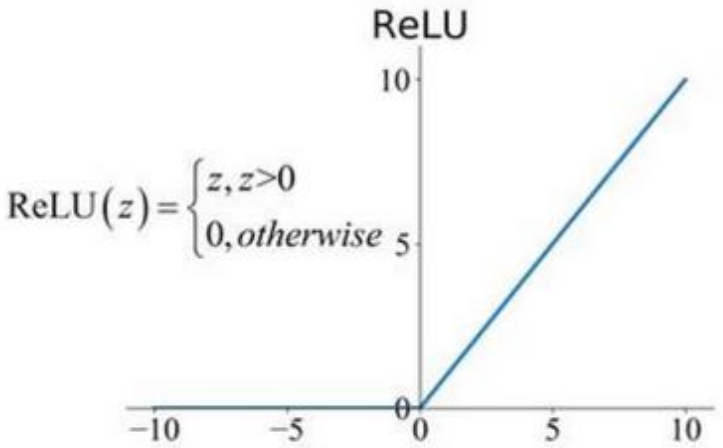
MLP: Backpropagation



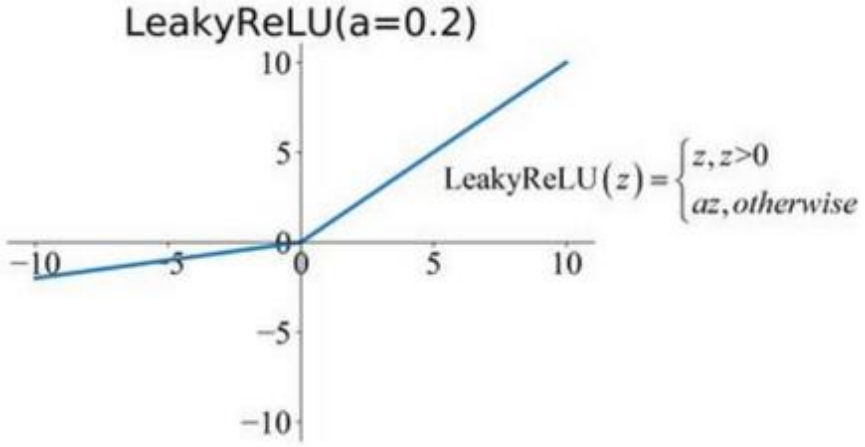
(a)



(b)

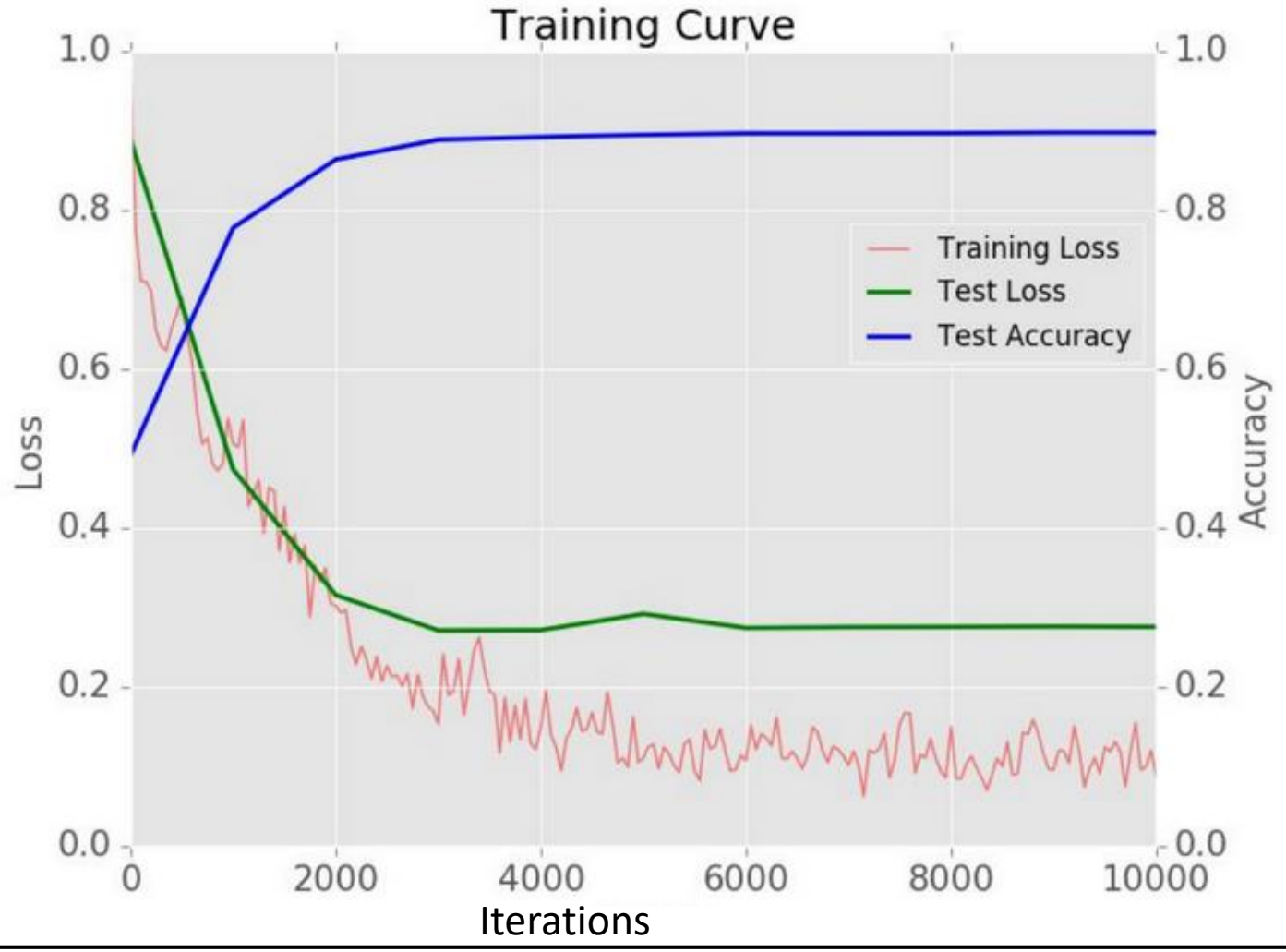


(c)



(d)

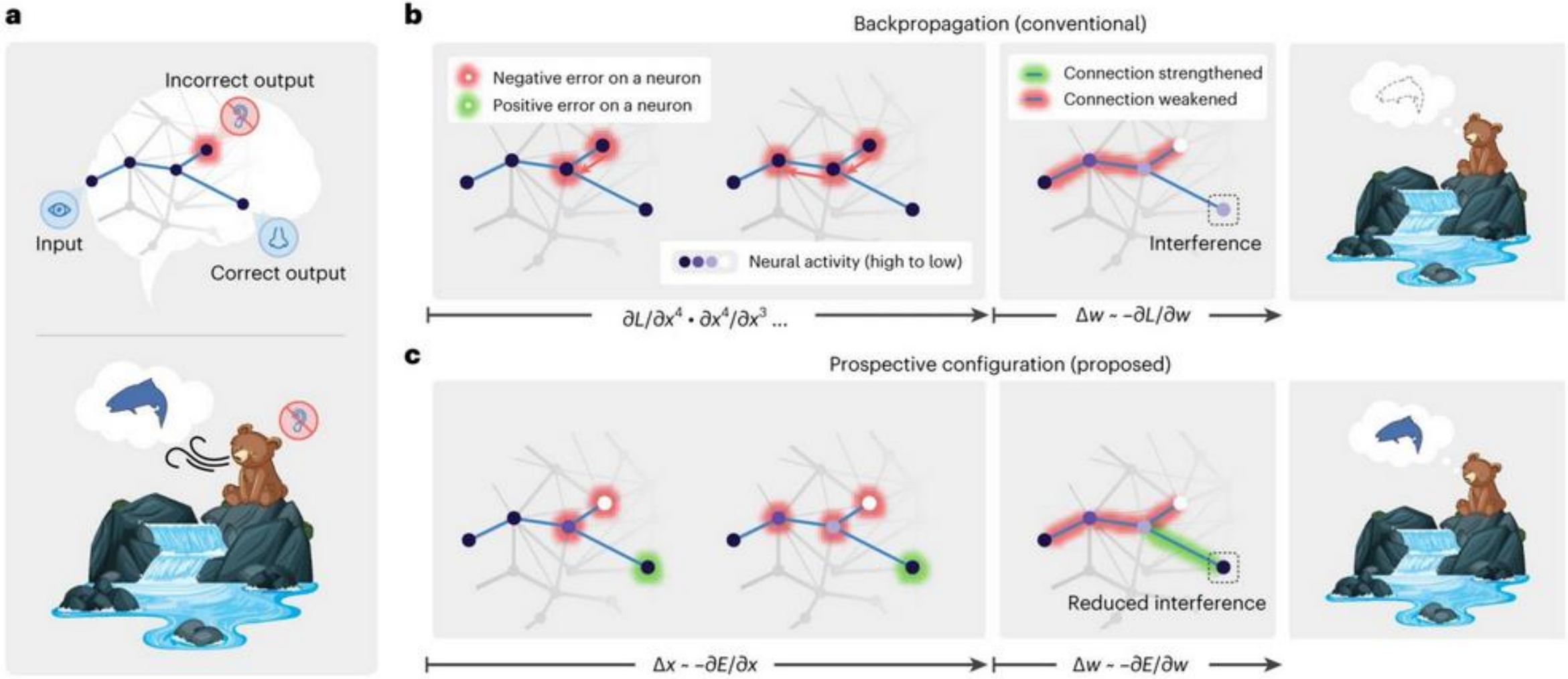
MLP: Processo Iterativo de Treino



A realidade é mais complexa

- O mecanismo de aprendizado de redes biológicas (cérebro) é diferente de ANNs
- Como aprendemos (repetição, visualização, associação... ?????)?
- nova “prospective configuration”
- **backpropagation > catastrophic interference**
- > aprender uma nova associação destrói as memórias previamente aprendidas

A realidade é mais complexa



A realidade é mais complexa

- O mecanismo de aprendizado de redes biológicas (cérebro) é diferente de ANNs
- Como aprendemos (repetição, visualização, associação... ?????)?
- nova “prospective configuration”
- **backpropagation > catastrophic interference**
- > aprender uma nova associação destrói as memórias previamente aprendidas

Créditos

- Prof. Fabio Alexandre SPANHOL, Dr.